

StockSense — Complete Product & Technical Documentation

Live Document — Updated automatically as the product evolves.

Last updated: 2026-06-24 (Session 9)

Table of Contents

1. [Product Overview](#)
 2. [Architecture Overview](#)
 3. [Data Sources](#)
 4. [Core Prediction Engine](#)
 5. [Technical Analysis Module](#)
 6. [Fundamental Scoring Module](#)
 7. [Sentiment Analysis Module](#)
 8. [Global Macro Context Module](#)
 9. [Quality Factors Module](#)
 10. [Risk Penalty Framework](#)
 11. [Confidence Engine](#)
 12. [Target Price & Trade Levels](#)
 13. [Daily Picks Engine](#)
 14. [Backtesting & Validation Engine](#)
 15. [Crypto Prediction Module](#)
 16. [Screener & Universe Management](#)
 17. [Paper Trading Module](#) 18a. [Reading the UI — Signal Colors & Common Jargon](#)
 18. [Alerts System](#)
 19. [API Reference](#)
 20. [Frontend Pages & Components](#)
 21. [Infrastructure & Deployment](#)
 22. [Automation Workflows](#)
 23. [Persistence & Data Durability](#)
 24. [Factor Weights by Horizon](#)
 25. [Key Design Principles](#)
 26. [Changelog](#)
-

1. Product Overview

StockSense is an AI-powered stock prediction and portfolio intelligence platform built for Indian and US equity markets. It combines institutional-grade quantitative signals with a consumer-friendly interface to deliver actionable BUY / HOLD / SELL signals with full explainability.

What StockSense Does

- Generates **BUY / HOLD / SELL** signals for Nifty 100, US large-cap, and top cryptocurrencies
- Delivers **Daily Picks** every morning at 9 AM IST — top 5 BUY ideas per horizon (short / medium / long)
- Shows **why** every signal was generated — factor breakdown, confidence scores, reasoning bullets
- Provides **trade levels** — entry zone, stop-loss, and target price with R:R ratio
- Runs a **learning engine** — tracks prediction outcomes and retrains factor weights weekly
- Supports **screener**, **backtest**, **watchlist**, and **alerts**

Supported Markets

Market	Universe	Horizons
India (NSE)	Nifty 100	Short (1–5 days), Medium (2–4 weeks), Long (3–6 months)
US	S&P 500 large-caps	Short, Medium, Long
Crypto	BTC, ETH, BNB, SOL, XRP, DOGE, ADA, AVAX, LINK, DOT	Short, Medium, Long

2. Architecture Overview



Tech Stack

Layer	Technology
Frontend	Next.js 14, React, TypeScript, TailwindCSS, Recharts
Backend	Python 3.11, FastAPI, Uvicorn
Auth	Supabase (JWT)
Database	PostgreSQL (production), SQLite (local)

Cache	In-memory (TTL-based), disk (picks_cache.json)
Hosting	Render.com (backend), Vercel (frontend)
Automation	GitHub Actions (cron jobs)

3. Data Sources

Source	Data Provided	Frequency	Used For
yfinance	Price, OHLCV, P/E, ROE, FCF, beta, analyst targets	Real-time	All markets
screener.in	10-year financials, ROCE, CAGR, promoter %, pledge %	Daily	India fundamentals
BSE API	Fundamentals for renamed / merged stocks	Daily	India fallback
NSE FII/DII API	Daily institutional flows (■ Cr)	Daily	India quality signal
NSE Pledge API	Promoter pledge % (quarterly disclosure)	Quarterly	India risk signal
Yahoo Finance RSS	News headlines per symbol	Real-time	Sentiment
Google News RSS	{symbol} stock India search results	Real-time	Sentiment fallback
Economic Times RSS	India economy & market news	Real-time	India sentiment
MoneyControl RSS	Stock & sector news	Real-time	India sentiment
yfinance macro	S&P 500, VIX, Crude, Gold, USD/INR, Nifty IT/Bank	15-min cache	Global macro

Data Fallback Chain (India)

yfinance → screener.in (if <5 key fields) → BSE API (if still sparse)

4. Core Prediction Engine

File: backend/services/prediction_engine.py

Signal Generation Formula

Composite Score = (Tech × W_tech) + (Fund × W_fund) + (Sent × W_sent) + Global Macro Adjustment + Analyst Consensus Adjustment + 52-Week Position Adjustment + Quality Factor Adjustment +

All component scores are on a **0–100 scale** (50 = neutral). The composite is also 0–100.

Signal Thresholds

Composite Score	Signal	Score Band	Confidence Calculation
≥ 90	BUY	Exceptional Opportunity	$(\text{score} - 60) / 40 \times 100\%$
≥ 75	BUY	Strong Buy Candidate	$(\text{score} - 60) / 40 \times 100\%$
≥ 60	BUY	Good Watchlist Stock	$(\text{score} - 60) / 40 \times 100\%$
45 – 59	HOLD	Neutral — Monitor	$50 - \text{abs}(\text{score} - 52) \times 2$
< 45	SELL	Avoid	$(45 - \text{score}) / 45 \times 100\%$

Dynamic Weights (Horizon × Volatility × Regime)

Base weights by horizon:

Horizon	Technical	Fundamental	Sentiment
Short (1–5 days)	70%	15%	15%
Medium (2–4 weeks)	40%	45%	15%
Long (3–6 months)	15%	75%	10%

Volatility modulation (applied on top of base weights):

Volatility Level	Annualised Vol	Technical	Fundamental
High	> 35%	Reduced to 10%	Boosted
Normal	15–35%	Base	Base
Low	< 15%	Boosted to 75%	Reduced

Regime modulation: - **BULL regime:** Boost technical, reduce fundamental - **BEAR regime:** Boost fundamental, reduce technical - **SIDEWAYS:** No modulation (enables mean-reversion trades)

Prediction Caching

- **TTL:** 15 minutes per (`symbol:market:horizon`) key
- **Max size:** 300 entries (Render 512 MB free-tier limit)
- **Eviction:** LRU — oldest entry dropped when full
- **Async pattern:** First request returns HTTP 202 (computing); client polls every 5s; result cached on completion

5. Technical Analysis Module

File: backend/services/technical_indicators.py

Indicators Computed

C	Indicator	Parameters
Momentum	Momentum	14-period
	Stochastic RSI	14-period
	Williams %R	14-period
	CCI	20-period
	Stochastic Oscillator	K=14, D=3
Trend	MACD	12/26/9 EMA
	EMA	20, 50, 200 periods
	ADX + DI+/DI-	14-period
	Bollinger Bands	20-period, 2 σ
Volatility	ATR	14-period
	Volume	Rolling
	Volume SMA	20-period
	VWAP	20-day rolling

Scoring Rules

RSI: - < 30 (oversold): **+15 pts** - 30–45 (recovering): **+7 pts** - 60–70 (elevated): **–7 pts** - > 70 (overbought): **–15 pts**

MACD: - Above signal line: **+12 pts** - Below signal line: **–12 pts**

EMA Trend: - Price > EMA200 (bull market structure): **+10 pts** - EMA20 > EMA50 (golden cross zone): **+8 pts** - EMA20 < EMA50 (death cross zone): **–8 pts**

ADX (Trend Strength): - ADX > 25 AND +DI > –DI (strong uptrend): **+10 pts** - ADX > 25 AND –DI > +DI (strong downtrend): **–10 pts** - ADX ≤ 25 (sideways): **0 pts**

Bollinger Bands (%B): - Near lower band (< 0.1): **+8 pts** (oversold) - Near upper band (> 0.9): **–8 pts** (overbought)

Volume Confirmation: - Up-day volume > down-day volume: **+8 pts** - Down-day volume > up-day volume: **–8 pts**

Candlestick Patterns:

Pattern	Points	Signal
Hammer	+5	BUY

Morning Star	+5	BUY
Bullish Engulfing	+5	BUY
Shooting Star	-5	SELL
Evening Star	-5	SELL
Bearish Engulfing	-5	SELL

Technical Signal Output

- Score $\geq$ 58: **BUY**
- Score $\leq$ 42: **SELL**
- 42–58: **HOLD**

6. Fundamental Scoring Module

File: backend/services/prediction_engine.py → `_fundamental_score()`

Quality Gate (Hard Rejection)

A stock is flagged **REJECTED** before scoring if **any** of these apply: - ROE < -10% (severely destroying shareholder value) - Profit Margin < -15% (deeply loss-making) - Non-positive Operating Cash Flow on medium/long horizons (core business not generating cash) - D/E ratio > 500% (extreme leverage, non-financial sector only)

Previously ROE AND margin both had to be negative simultaneously — this was too lenient and has been corrected.

Scoring Architecture (Per-Category Budgets)

The fundamental score uses **six independent capped buckets**, each scored separately, then summed with a base of 50. This prevents any single dimension from dominating and ensures the final score has meaningful discrimination across the full 0–100 range.

Bucket	Cap	What It Measures
Valuation	±15	P/E, P/B ratios vs market-calibrated thresholds
Profitability	±15	ROE, ROCE, profit margins
Growth	±15	Revenue + earnings CAGR (counted once via longest available window)
Balance Sheet	±10	D/E, OCF quality, Altman Z-Score, Sloan accruals
Governance	±10	Promoter holding, FII/DII flows, promoter pledge
Banking	±10	Net NPA, NIM (fires only for banks/NBFCs)

Total possible range: $50 \pm 65 \rightarrow$ clamped to $[0, 100]$

Valuation bucket (cap ± 15)

Metric	Threshold (India / US)	Points
P/E	< 18 IN / < 15 US (cheap)	+8
P/E	< 30 IN / < 25 US (fair)	+3
P/E	> 55 IN / > 50 US (expensive)	−8
P/B	< 2.5 IN / < 2.0 US	+4
P/B	> 8.0 IN / > 6.0 US	−4

Profitability bucket (cap ± 15)

Metric	Threshold	Points
ROE	> 20%	+7
ROE	10–20%	+3
ROE	< 0%	−7
Profit Margin	> 20%	+5
Profit Margin	< 0%	−5
ROCE	> 20%	+6
ROCE	12–20%	+2
ROCE	< 6%	−4

Growth bucket (cap ± 15) — revenue and earnings each counted once

Revenue uses 3Y CAGR (screener.in) if available, else TTM YoY (yfinance):

Metric	Threshold	Points
3Y Revenue CAGR	> 15%	+7
3Y Revenue CAGR	8–15%	+3
3Y Revenue CAGR	< 0%	−5
TTM Revenue Growth (fallback)	> 20%	+7
TTM Revenue Growth	5–20%	+3
TTM Revenue Growth	< −5%	−5

Earnings uses longest available: 5Y CAGR (long horizon) $\rightarrow$ 3Y CAGR $\rightarrow$ TTM EPS growth:

Metric	Threshold	Points
--------	-----------	--------

5Y Profit CAGR (long only)	> 18%	+6
3Y Profit CAGR	> 20%	+6
3Y Profit CAGR	> 10%	+3
3Y Profit CAGR	< -10%	-5
TTM EPS Growth (fallback)	> 20%	+5
TTM EPS Growth	< -10%	-5
Quarterly PAT trend	Accelerating	+3
Quarterly PAT trend	Decelerating	-3

Balance Sheet bucket (cap ± 10)

Metric	Threshold	Points
D/E	> 300%	-7
D/E	150-300%	-3
D/E	< 50%	+3
Operating CF (screener)	Negative	-5
Operating CF 3Y growth	> 30%	+4
Operating CF 3Y growth	Positive	+2
Altman Z-Score	Safe zone (medium/long)	+3
Altman Z-Score	Grey zone	-4
Altman Z-Score	Distress zone	-8
Sloan Accruals ratio	< -5% (cash-backed)	+3
Sloan Accruals ratio	> 10% (manipulation risk)	-5

Governance bucket (cap ± 10) — India only

Metric	Threshold	Points
FII + DII combined	> 50%	+4
FII + DII combined	25-50%	+2
DII quarterly trend	Up > 3% (MF accumulation)	+3
DII quarterly trend	Down > 3%	-3
FII quarterly trend	Up > 3%	+2
FII quarterly trend	Down > 3%	-2
Promoter holding	> 55%	+2

Promoter holding	< 25%	-2
Promoter trend	Up > 2% (insider buying)	+3
Promoter trend	Down > 3% (insider selling)	-4
Promoter pledge	> 50%	-8
Promoter pledge	25-50%	-5
Promoter pledge	10-25%	-2
Promoter pledge	0%	+2

Banking bucket (cap ±10) — fires only for banks/NBFCs

Metric	Threshold	Points
Net NPA	> 3%	-7
Net NPA	1.5-3%	-3
Net NPA	< 0.5%	+4
NIM	> 4%	+4
NIM	< 2%	-3

7. Sentiment Analysis Module

File: backend/services/news_sentiment.py

Two-Layer Scoring

Layer 1 — Financial Lexicon (60% weight): - ~80 domain-specific phrases with pre-calibrated scores - Examples: "beat expectations" → +0.75, "profit warning" → -0.80, "upgrade" → +0.65 - Designed to override generic sentiment on financial language

Layer 2 — VADER Sentiment (40% weight): - Title sentiment (70%) + Description sentiment (30%) - Standard NLP library tuned for social media / news

Final blend: $\text{score} = 0.60 \times \text{lexicon} + 0.40 \times \text{VADER}$

News Sources (per prediction)

Source	Articles	Priority
Yahoo Finance RSS ({symbol}, {symbol}.NS)	10	Primary
Google News RSS ({symbol} stock India)	10	Secondary
Economic Times RSS	10	India supplement

MoneyControl RSS	10	India supplement
------------------	----	------------------

Cache TTL: 10 minutes per symbol

Sentiment Classification

Score	Label
≥ +0.05	BULLISH
−0.05 to +0.05	NEUTRAL
≤ −0.05	BEARISH

Score conversion: `sentiment_score = 50 + (bullish − bearish) / (bullish + bearish) × 50`

Neutral articles are excluded from the denominator. Previously neutrals diluted the score — 5 bullish + 5 neutral incorrectly scored the same as 5 bullish + 5 bearish. Now only labelled articles (bullish + bearish) count.

When no news available: Returns neutral (50), redistributes weight to technical + fundamental, sets `data_available = False` flag.

8. Global Macro Context Module

File: `backend/services/global_context.py`

Macro Indicators Tracked

Indicator	Ticker	What It Signals
S&P 500	<code>^GSPC</code>	US demand → IT/pharma export tailwind
NASDAQ	<code>^IXIC</code>	Tech hiring, demand for Indian IT services
VIX	<code>^VIX</code>	FII flows: high VIX → FII outflows from India
USD/INR	<code>INR=X</code>	Weak INR → pharma/IT revenue boost; import cost rise
Crude Brent	<code>BZ=F</code>	Oil import cost, OMC margins
Gold	<code>GC=F</code>	Jewelry demand (Titan, Muthoot)
Nifty IT Index	<code>^CNXIT</code>	Sector momentum for IT stocks
Nifty Bank Index	<code>^NSEBANK</code>	Banking sector rotation signal

Cache TTL: 15 minutes (fetched in parallel)

Stock-Specific Macro Sensitivity Map

Over 100 Nifty stocks mapped to their macro sensitivities:

Stock Category	Tailwind Factors	Headwind Factors
IT (TCS, INFY, WIPRO, HCL)	USD/INR weakness, S&P 500 up, NASDAQ up	INR strengthening
Pharma (SUNPHARMA, DRREDDY)	USD/INR weakness, S&P 500 up	—
OMCs (BPCL, IOC)	Crude down	Crude up (margin squeeze)
Oil producers (ONGC)	Crude up	Crude down
Paints (ASIANPAINT, PIDILITIND)	—	Crude up (TiO2 input cost)
Banks (HDFCBANK, ICICIBANK)	—	VIX up (FII sensitivity)
Jewelry (TITAN)	Gold stable	Gold up (input cost)

Stock Adjustment Calculation

```
stock_adj = Σ sensitivity_i × (factor_i – benchmark_i) Tailwind factors: +2 to +4 points
Headwind factors: -2 to -4 points
```

9. Quality Factors Module

File: backend/services/quality_factors.py

Applied for India (Nifty 100) only, on medium and long horizons.

10 Dimensions of Quality

1. Earnings Revisions (Weight: 12–14%)

- EPS surprise trend (beat vs miss last 4 quarters): ±16 pts
- Analyst upgrade/downgrade momentum: ±8 pts
- Forward PE compression vs trailing PE: ±8 pts

2. Institutional Ownership (Weight: 5–6%)

- Holdings > 50%: +14 pts
- Holdings 30–50%: +8 pts
- Holdings 15–30%: +3 pts
- Holdings < 5%: -5 pts
- Institution count > 300: +6 pts; < 20: -4 pts

3. Institutional Flow Proxy / MF Trend (Weight: 5–13%)

Blends price-volume signals (OBV trend, MFI, accumulation pattern) with real NSE FII/DII data: - 60% price-volume proxy + 40% real FII/DII flows (when available)

4. Relative Strength (Weight: 7–15%)

Stock return vs NIFTY 50 over 1M, 3M, 6M: - Outperform > 10%: +12 pts - Outperform 4–10%: +6 pts - Underperform > 10%: –12 pts - Underperform 4–10%: –6 pts

5. Sector Strength (Weight: 7–15%)

Sector index momentum vs NIFTY 50: - Sector outperform > 5%: +16 pts - Sector outperform 2–5%: +8 pts - Sector underperform > 5%: –14 pts - Sector underperform 2–5%: –7 pts

6. Valuation Quality (Weight: 8–17%)

Multi-dimensional valuation scoring: - PEG < 0.75: +16 pts; > 2.5: –12 pts - EV/EBITDA vs sector: ±10 pts at major discount/premium - Sector-relative PE: ±12 pts at 30% discount - P/B for banks < 1.0: +12 pts; > 4.0: –8 pts - Analyst target upside > 30%: +10 pts (margin of safety)

7. Risk Management (Weight: 10%)

- Max Drawdown 12M < –10%: +14 pts; < –40%: –14 pts
- Volatility Percentile bottom 25%: +10 pts; top 20%: –10 pts
- Sharpe Ratio > 1.5: +12 pts; < 0: –10 pts
- Downside Deviation < 10%: +6 pts; > 30%: –6 pts

8. Corporate Actions (Weight: 3–10%)

- Dividend payer 5Y+: +8 pts
- Growing dividends: +6 pts
- Payout ratio 0–40%: +4 pts; > 80%: –6 pts
- Active buyback: +8 pts
- Share dilution: –5 pts

9. Liquidity (Weight: 4–8%)

- Market cap ■2T+: +10 pts; < ■10B: –8 pts
- Avg daily volume > 5M: +8 pts; < 100K: –8 pts
- Beta 0.5–1.2: +4 pts; > 2.0: –5 pts

10. Quality Metrics (Weight: 4–12%)

Piotroski F-Score (9-point scale):

Point	Criterion
P1	ROA > 0
P2	Operating Cash Flow > 0
P3	ROA improving YoY
P4	Accruals: cash earnings > accrual earnings
P5	Leverage decreasing
P6	Current ratio improving

P7	No share dilution
P8	Gross margin expanding
P9	Asset turnover improving

- Score ≥ 8 : +20 pts
- Score 6–7: +10 pts
- Score 4–5: 0 pts
- Score < 4 : –12 pts

ROIC (Return on Invested Capital): - $> 20\%$: +14 pts - 12–20%: +8 pts - 6–12%: +2 pts - $< 6\%$: –8 pts

Horizon-Based Weighting

Dimension	Short	Medium	Long
Earnings Revisions	13%	14%	12%
Institutional Ownership	5%	6%	6%
MF/FII Flow	6%	8%	10%
Relative Strength	15%	11%	7%
Sector Strength	15%	11%	7%
Valuation Quality	8%	13%	17%
Risk Management	10%	10%	10%
Corporate Actions	3%	6%	10%
Liquidity	8%	6%	4%
Quality Metrics	4%	6%	12%
Flow Proxy	13%	9%	5%

10. Risk Penalty Framework

Applied **after** all signal scoring. Never adds points — only subtracts (risk override).

Risk Factor	Condition	Penalty
High leverage	D/E $> 300\%$	–8 pts
Elevated leverage	D/E 200–300%	–4 pts
High beta	Beta > 2.0	–6 pts
Elevated beta	Beta 1.6–2.0	–3 pts
Negative FCF	FCF < 0	–5 pts

Negative ROE	ROE < 0	–5 pts
Poor risk profile	Risk score < 35	–5 pts
Earnings volatility	CV > 0.5	–4 pts

Maximum penalty capped at –30 pts (prevents extreme scores on genuinely bad risk/reward stocks).

11. Confidence Engine

Answers: "How much should you trust this signal?"

Five Components

Component	What It Measures	Weight (Full)	Weight (Bootstrap)
Data Completeness	% of key fundamental fields present	25%	31.25%
Factor Agreement	% of factors agreeing with signal direction	25%	31.25%
Earnings Stability	Quality earnings_revision sub-score	15%	18.75%
Regime Certainty	BULL/BEAR vs SIDEWAYS trend strength	15%	18.75%
Historical Factor Reliability	Live IC values (needs 60+ outcomes)	20%	0% → fallback 50

Sector-Aware Field Sets

Indian Non-Financial: PE, ROE, revenue growth, D/E, margin, beta (yfinance) + 3Y CAGR, ROCE, FII, promoter (screener) = 10 fields

Indian Financial (banks/NBFCs): PE, ROE, revenue growth, profit margin, EPS growth, beta (yfinance) + 3Y CAGR, FII, promoter (screener) = 10 fields

US Stocks: PE, ROE, revenue growth, D/E, margin, EPS growth, FCF, beta, ROCE = 13 fields

Confidence Bands

Score	Band
≥ 80	HIGH
60–79	MEDIUM
< 60	LOW

Percentile Context

Score	Label
≥ 80	Top 10% of all Nifty predictions
≥ 72	Top 20%
≥ 65	Top 35%
≥ 58	Top 50%
≥ 50	Below average
< 50	Low range

12. Target Price & Trade Levels

File: backend/services/prediction_engine.py → `_target_price()`, `_trade_levels()`

Target Price

Horizon	Method
Short	ATR × 2.5 × confidence factor — moves 1–5 day magnitude
Medium	Analyst target (70%) + price projection (30%); BUY floor: <code>max(blend, price×1.05)</code>
Long	<code>analyst_target × (1+EPS_growth)²</code> ; BUY floor: <code>max(target, price×1.15)</code>

Trade Levels

Take Profit = Model Target Price (consistent with signal) Stop Loss = ATR-based, adjusted to maintain minimum R:R of 1.5× Entry Zone = [price – 0.3×ATR, price + 0.1×ATR] for BUY [price – 0.1×ATR, price + 0.3×ATR] for SELL R:R Ratio = (target – price) / (price – stop_loss)

ATR multipliers by horizon:

Horizon	Base SL	Floor SL	Max SL
Short	1.5× ATR	1× ATR	25% of price
Medium	3× ATR	2× ATR	25% of price
Long	5× ATR	3× ATR	25% of price

13. Daily Picks Engine

File: backend/services/daily_picks.py

Execution Schedule

- Triggered: **Every weekday at 9:00 AM IST** (via GitHub Actions → POST `/api/picks/generate`)
- Generation time: ~10 minutes
- Results cached to disk: `backend/picks_cache.json`
- API response: Instant (reads from cache)

9-Phase Pipeline

Phase 0 — Outcome Resolution

- Compare previous predictions against actual forward returns (1-day / 63-day / 252-day)
- Update outcome logger database with direction hits (correct/incorrect)
- Feed data into IC engine for retraining

Phase 1 — Universe Screening

- Run full prediction engine on all Nifty 100 stocks
- Parallelised: 2 workers via `ThreadPoolExecutor`
- Returns raw factor scores for all stocks (enables cross-sectional z-scoring)

Phase 2 — Regime Detection

- KMeans clustering (4 clusters) on global macro features: VIX, S&P 500 return, crude, gold, USD/INR
- Classifies market into: **BULL_CALM**, **BULL_VOLATILE**, **BEAR_CALM**, **BEAR_PANIC**
- Returns regime label + weight multipliers for IC adjustment

Phase 3 — IC Weight Computation

- If < 60 outcome pairs: use academic prior weights
- Short: tech=0.055, fund=0.018, sentiment=0.042, quality=0.032
- If ≥ 60 pairs: Bayesian shrinkage blend of live IC + prior:
- $\text{weight} = \text{live_weight} \times \text{live_ic} + (1 - \text{live_weight}) \times \text{prior}$
- Apply regime multipliers: BULL boosts tech/sentiment; BEAR boosts fundamental/quality

Phase 4 — Z-Score Normalisation & Alpha

- Cross-sectional z-scoring: $z_i = (\text{score}_i - \text{mean}) / \text{std}$
- Combined alpha: $\sum \text{ic_weight}_k \times z_k$
- Meta-model alpha (requires 180+ outcomes across 60 stocks × 3 horizons):
- Inputs: tech_z, fund_z, sentiment_z, quality_z, combined_alpha, regime_id
- Output: predicted return %

Phase 5 — Pick Selection

- Rank by alpha score (meta_alpha if available, else combined_alpha)
- Select top 5 **BUY** signals per horizon (composite score ≥ 60)
- Minimum 1 pick per horizon
- Empty picks from a prior run (0 BUY signals) are NOT treated as "complete" — startup catch-up will retry on next deploy

Phase 6 — Portfolio Optimisation

- Fetch 6-month daily returns for selected picks
- Covariance estimation: Ledoit-Wolf shrinkage at 25%
- Optimise: $\max (\alpha \times w - \lambda \times w^T \Sigma w)$ via SLSQP
- Constraints: $\Sigma w = 1.0, 0 \leq w_i \leq 0.40$ (max 40% per position)
- Risk aversion λ : doubled in BEAR_PANIC regime
- Fallback (if scipy unavailable): iterative alpha-proportional weights that correctly enforce the 40% cap

Phase 7 — Logging

- Log to PostgreSQL (if `USE_POSTGRES=1`) or SQLite
- Stored fields: factor z-scores, combined_alpha, meta_alpha, signal, price, horizon, regime

Phase 8 — Weight Adaptation (background)

- Retrain IC engine with new outcomes
- Update meta-model if sufficient data
- Recalibrate regime clustering
- Runs in daemon thread (non-blocking)

14. Backtesting & Validation Engine

File: `backend/services/validation_engine.py`, `backend/services/backtester.py`

Walk-Forward Methodology

For each business day t in Nifty 100 history: 1. Fetch price data available **only before** time t 2. Compute prediction at t using that data 3. Measure actual forward return at $t + h$ ($h = 7 / 63 / 252$ days) 4. Compare predicted signal vs actual direction

Known limitation (next session): Technical indicators (EMA, MACD, OBV) are currently computed on the full historical DataFrame before slicing per date — a form of look-ahead bias in the backtester. This inflates reported validation hit rates. A full fix (recompute indicators per window) is planned for Session 5.

Metrics Computed

Metric	Definition
Hit Rate	% of BUY/SELL calls that were directionally correct
Avg Return on BUY	Mean forward return when signal was BUY
Sharpe Ratio	$(\text{avg_return} - \text{risk_free}) / \text{std_return}$ on BUY calls
vs Benchmark	Alpha over NIFTY 50 buy-and-hold
Score Calibration	Hit rate by score bucket (60–70, 70–80, 80–100)

Execution

- **Weekly:** Sunday 7:30 AM IST (via GitHub Actions)
- **Duration:** ~40 minutes (medium + long horizons)
- **Storage:** PostgreSQL `val_runs` + `val_signals` tables
- **Retention:** 365 days

Forward Return Windows

Horizon	Forward Return Used	Outcome Resolution Wait
Short	return_5d (5 trading days)	3 calendar days
Medium	return_20d (20 trading days ≈ 1 month)	30 calendar days
Long	return_60d (60 trading days ≈ 3 months)	90 calendar days

Partial returns are never logged — the outcome logger returns None if fewer than the required trading days have elapsed. This prevents truncated returns from contaminating IC training data.

Validation BUY Threshold

Validation uses the same threshold as the live prediction engine: **composite** ≥ **60** = **BUY** for all horizons. Previously the thresholds were mismatched (validation used 65, live used 60), making validation metrics unmeasurable against the actual model.

Indicative Score Calibration (Nifty 100, Medium-Term)

Score Bucket	Hit Rate	Beat Benchmark	Avg Alpha
80–100 (Exceptional/Strong BUY)	~68%	~62%	+4.2%
75–79 (Strong BUY)	~62%	~55%	+2.8%
60–74 (Good Watchlist BUY)	~58%	~50%	+1.5%
45–59 (HOLD)	~52%	~48%	+0.3%

Note: These figures reflect the post-Session 4 thresholds. Historical validation data accumulated under the old 70-threshold is being re-calibrated.

15. Crypto Prediction Module

File: backend/services/crypto_engine.py

Supported Assets

BTC, ETH, BNB, SOL, XRP, DOGE, ADA, AVAX, LINK, DOT

Signals Used (No Fundamentals)

Signal	Weight
Technical indicators (full suite)	Primary
Fear & Greed proxy (30-day vol / 90-day vol)	Secondary
On-chain proxy (price-volume accumulation)	Secondary
News sentiment (Google News + CoinTelegraph)	Supporting
Macro sensitivity (BTC/ETH correlated to S&P, VIX)	Adjustment

Quality factors and dynamic fundamental weights are not applied — crypto lacks P/E, ROE, cash flow data. Momentum and sentiment dominate.

16. Screener & Universe Management

File: backend/services/screener_service.py, screener_data.py

Screener Filters

Filter	Type	Description
Market	US / IN	Filter by market
Signal	BUY / HOLD / SELL	Filter by signal
Min Market Cap	■ Cr	Size filter
Max P/E	Ratio	Valuation filter
Min ROE	%	Profitability filter
Sector	IT / Banking / Pharma / etc.	Sector filter

Heatmap

- Groups stocks by sector; sorted by sector avg change% (best sectors on top)
- **India: 25 sectors** — Banking, IT, Auto, Pharma, Energy, FMCG, Finance, Healthcare, Insurance, Chemicals, Cement, Metal & Mining, Defence, Realty, Telecom, Consumer Disc, Hotels & Travel, Food & Beverage, Media & Entmt, Textiles, Agro & Chemicals, Logistics, Paints, Infra, Capital Goods, Power, EV & New Energy
- **US: 29 sectors** — Mega Cap Tech, Semiconductors, Cloud & SaaS, Cybersecurity, Fintech, Finance, Insurance, Healthcare, Biotech, Med Devices, Energy, Clean Energy, EV, Consumer Disc, Consumer Stap, E-commerce, Social Media, Streaming & Media, Gaming, Aerospace & Defence, Industrials, Airlines, Cruise & Hotels, Restaurants, Retail, Telecom, Utilities, Realty, Materials, Crypto & Blockchain
- Up to **15 stocks per sector** (MAX_STOCKS = 15)
- Primary data source for India sectors: **NSE sector index APIs** (SECTOR_TO_NSE_INDEX mapping); Yahoo Finance used as fallback for symbols not in NSE indices and as primary source for all US sectors

- Grey tiles = symbol has no live data; all 353 India symbols audited and bad tickers corrected (Session 5)
- Colour-coded by performance (green/red intensity); loading status badge (Fetching / Refreshing / Live)

Top Movers

- Fetched in real-time from yfinance
- Ranked by absolute % change
- Separate for US, India, Crypto

17. Paper Trading Module

File: backend/api/routers/paper_trading.py, frontend/src/app/paper-trading/page.tsx

A simulated trading environment where users can test stock calls without real money. All trades persist in PostgreSQL and survive Railway restarts.

Database Schema

```
paper_portfolio (user_id TEXT PK, session_id TEXT, cash NUMERIC, updated_at TIMESTAMPTZ)
paper_trades (id SERIAL PK, user_id TEXT, session_id TEXT, symbol TEXT, market TEXT, quantity
INT, entry_price NUMERIC, exit_price NUMERIC, stop_loss NUMERIC, target_price NUMERIC, status
TEXT, signal TEXT, horizon TEXT, opened_at TIMESTAMPTZ, closed_at TIMESTAMPTZ)
```

User Model

- Trades are scoped to **Supabase user_id** (stable UUID from `useAuth().user.id`)
- No more session_id / localStorage dependency — trades persist across all browsers and devices
- Starting virtual cash: **■1,00,000 / \$10,000** (depending on market)
- All positions are long only (no shorting)

Trade Lifecycle

```
Open Trade → entry_price captured at trade time, keyed to user_id Live Price → fetched
real-time via yfinance / NSE Unrealised P&L = (live_price - entry_price) × quantity Close Trade
→ exit_price set, status = 'CLOSED' Realised P&L = (exit_price - entry_price) × quantity
```

Stop Loss & Target Price

- Optionally set per trade via inline edit (↵ icon)
- ATR-based defaults pre-filled when placing a trade from the stock detail page
- Both values always shown per position with % from entry
- UI highlights rows where price is within 2% of stop (yellow) or target (green)
- The Buy button is **not blocked** by AI prediction loading — trade is placeable immediately; stop loss/target suggestions update in the background

API Endpoints

Endpoint	Method	Description
/api/paper-trading/portfolio	GET	Portfolio summary (pass user_id as query param)
/api/paper-trading/buy	POST	Open a new position (body: user_id, symbol, market, qty, price, ...)
/api/paper-trading/sell/{trade_id}	POST	Close a position (body: user_id, price)
/api/paper-trading/trade/{trade_id}	PATCH	Edit stop_loss / target_price (body: user_id, ...)
/api/paper-trading/reset	POST	Reset portfolio to starting cash (query: user_id)

18. Portfolio Tracker

File: backend/api/routers/portfolio.py, frontend/src/app/portfolio/page.tsx

Manually-tracked real (or planned) holdings with live P&L — distinct from Paper Trading's AI-signal-driven simulated trades. Added in Session 8 (2026-06-23); previously stored entirely in the browser's `localStorage` with no cross-device sync, migrated to Postgres so a holding added on one device is visible on any other device for the same account. `localStorage` is now only a fast-access cache / offline fallback; on first load after the migration shipped, any pre-existing local-only holdings are pushed up to the server automatically.

Database Schema

```
portfolio_holdings (id TEXT PK, user_id TEXT, symbol TEXT, market TEXT CHECK('IN','US'), qty NUMERIC, avg_price NUMERIC, created_at TIMESTAMPTZ)
```

API Endpoints

Endpoint	Method	Description
/api/portfolio/{user_id}	GET	All holdings for user
/api/portfolio/{user_id}	POST	Add a holding (body: symbol, market, qty, avg_price)
/api/portfolio/{user_id}/{holding_id}	PATCH	Edit qty / avg_price
/api/portfolio/{user_id}/{holding_id}	DELETE	Remove holding

Frontend Behavior

- IN and US holdings render in separate sections (never mixed — ■ and \$ totals can't be summed into one number).
- Inline edit (pencil icon) for Qty/Avg Buy, confirmed via checkmark or Enter, cancelled via X or Escape.
- Delete/edit await the backend response before updating local state — closes the same "resurrection" race class found in Alerts (a failed request used to leave the row alive server-side while the UI showed it changed/gone, with

the next page load silently reverting it).

- Symbol entry uses the shared `StockSymbolField` predictive-search component (Session 8) instead of a bare text input.

18a. Reading the UI — Signal Colors & Common Jargon

A reference for what the colors, badges, and terms scattered across Portfolio, Paper Trading, Daily Picks, and Multibagger actually mean. Source of truth: `frontend/src/components/SignalBadge.tsx`.

Signal badge colors

Badge	Color	Meaning
BUY	Green	Confidence $\geq 60\%$ — strong conviction
BUY	Gold/yellow (filled)	Confidence 45–59% — moderate conviction
BUY	Gray (outlined)	Confidence $< 45\%$ — technically a BUY call, but weak conviction
HOLD	Amber/gold	Composite score landed in the neutral 45–59 band — model found no clear edge either way
REJECTED	Amber/gold (same color as HOLD, different label)	The model never scored this stock at all — a hard quality gate rejected it before scoring, typically for severe fundamental red flags or data quality issues. Different from HOLD: HOLD means "scored and neutral," REJECTED means "refused to score." Check that stock's <code>rejection_reasons</code> for specifics.
SELL	Red	Composite score below the SELL threshold
— (dash)	—	Not "no signal exists" — the row's data is still queued behind other rows in a staggered batch (added to avoid overwhelming the browser's per-origin connection limit on pages with many holdings/trades). It resolves to a real badge once its turn comes up; if it never resolves after a minute or more, that's worth reporting as a bug.

Color tokens: `bull` = green (`#22c55e`), `bear` = red (`#ef4444`), `neutral` = amber (`#f59e0b`). BUY is the only signal with confidence-graded shading; HOLD/REJECTED/SELL are always a single fixed shade regardless of confidence.

Other terms that show up across the app

Term	Where seen	Meaning
------	------------	---------

Confidence %	Signal badges, prediction detail	How sure the model is about its own call — a measure of conviction, not a probability of being "right." Low confidence ≠ wrong, it means the underlying signals were mixed/weak.
Invested	Portfolio, Paper Trade	$Qty \times Avg\ Buy\ Price$ (or entry price for paper trades) — what you actually put in, unaffected by current price moves.
Unr. P&L	Paper Trade	"Unrealized P&L" — the gain/loss on an open position if you closed it right now at the current market price. Becomes "Realized P&L" only after you actually close the trade.
Near stop loss	Paper Trade	The current price has moved within a small buffer of the stop-loss price — a warning to consider closing, not an automatic close (this app has no real broker execution; stop-loss/target are advisory levels you act on manually).
Mkt Price (last close when closed)	Paper Trade	For OPEN trades this is the live price; for CLOSED trades it's frozen at whatever the price was at the last market close before/at closing, since a closed position has no further "live" price to track.
Target Price	Predictions, Paper Trade	The model's projected price for the selected horizon if the thesis plays out — not a guarantee, and not the same as the stop-loss.
Horizon (short/medium/long)	Everywhere	short = 1–10 days, medium = 1–3 months, long = 6 months–3 years. Each horizon re-weights the same underlying factors differently (see §9 Dynamic Weights) — the same stock can legitimately show different signals across horizons at the same time.
Composite Score	Prediction detail, score history	The single 0–100 number all the BUY/HOLD/SELL thresholds are based on — a weighted blend of technical, fundamental, sentiment, macro, and quality factors (see §4).
Verdict (Multibagger)	Multibagger screen	<code>strong_buy</code> / <code>watchlist</code> / <code>avoid</code> — a checklist-based pass/fail rating distinct from the BUY/HOLD/SELL prediction signal; a stock can be a Multibagger "watchlist" candidate while simultaneously showing a HOLD prediction signal, since they're answering different questions (long-term quality checklist vs. current-horizon timing call).

Shortlisted (Multibagger)	Multibagger screen	The top ~20% scorers within a screen — a relative ranking marker, not a guarantee.
REJECTED reasons	Multibagger, Predictions	A specific list of which hard-gate checks failed (e.g. promoter pledge too high, negative equity) — click through if a stock you expected to see is missing or rejected.

19. Alerts System

File: backend/api/routers/alerts.py, backend/services/price_alert_notifier.py, frontend/src/app/alerts/page.tsx

Price-level alerts that trigger when a stock crosses a target price. All alerts persist in PostgreSQL.

Database Schema

```
price_alerts (id TEXT PK, user_id TEXT, symbol TEXT, market TEXT, email TEXT, target_price
NUMERIC, direction TEXT CHECK('above','below'), triggered BOOL DEFAULT FALSE, created_at
TIMESTAMPTZ, triggered_at TIMESTAMPTZ)
```

Alert Check Logic

- **Client-side:** frontend polls live quote every 5 seconds while the page is open; on trigger, PATCHes the alert to triggered = true. Stops working the moment the tab is closed/backgrounded.
- **Server-side (Session 8):** `_price_alerts_check_loop` in `main.py` runs `price_alert_notifier.check_and_notify()` every 90 seconds — scans all non-triggered alerts with an email on file, fetches the live quote, and emails the owner (same Resend account as invites/paper-trade notifications) once the threshold is crossed, marking it triggered server-side. No time-based cooldown needed — triggered itself is the dedup, since the query only selects non-triggered rows. Kill switch: set `PRICE_ALERTS_ENFORCEMENT=0` to disable just this background check without a code change; client-side polling is unaffected either way.
- Triggered alerts shown with timestamp; can be reset (re-arms for exactly one more notification) or deleted.
- Delete/reset await the backend response before updating local UI state (Session 8 fix) — previously a failed request could leave the row alive server-side while the screen showed it changed, with the next page load silently reverting it.

API Endpoints

Endpoint	Method	Description
<code>/api/alerts/{user_id}</code>	GET	All alerts for user
<code>/api/alerts/{user_id}</code>	POST	Create new alert
<code>/api/alerts/{user_id}/{alert_id}</code>	PATCH	Update (reset triggered, edit price)
<code>/api/alerts/{user_id}/{alert_id}</code>	DELETE	Remove alert

20. API Reference

Prediction

Endpoint	Method	Description
/api/predictions/{symbol}	GET	Get prediction (async, 202 while computing)
/api/predictions/debug/state	GET	Internal cache/thread state (debug only)

Query params: `market` (US/IN/CRYPTO), `horizon` (short/medium/long)

Daily Picks

Endpoint	Method	Description
/api/picks/daily	GET	Today's cached picks (instant)
/api/picks/generate	POST	Trigger pick generation (secret-protected)
/api/picks/performance	GET	Live P&L of past picks vs benchmark

Query params for performance: `horizon` (short/medium/long), `window_days` (default 90)

Screener

Endpoint	Method	Description
/api/screener/filter	GET	Filter universe by fundamentals
/api/screener/heatmap	GET	Sector heatmap
/api/screener/top-movers	GET	Top 10 movers by % change
/api/screener/crypto-movers	GET	Top 10 crypto movers

Backtest & Validation

Endpoint	Method	Description
/api/backtest/run	POST	Single-stock walk-forward backtest
/api/validation/run	POST	Full universe validation (background)
/api/validation/status	GET	Validation progress
/api/validation/results	GET	Validation metrics by horizon

Watchlist

Endpoint	Method	Description
/api/watchlist/{user_id}	GET	All saved stocks
/api/watchlist/{user_id}	POST	Add stock
/api/watchlist/{user_id}/{symbol}	DELETE	Remove stock

Alerts

Endpoint	Method	Description
/api/alerts/{user_id}	GET	All price alerts
/api/alerts/{user_id}	POST	Create alert
/api/alerts/{user_id}/{alert_id}	PATCH	Update alert (reset / edit)
/api/alerts/{user_id}/{alert_id}	DELETE	Delete alert

Paper Trading

Endpoint	Method	Description
/api/paper-trading/portfolio	GET	Portfolio summary (?user_id=)
/api/paper-trading/buy	POST	Open position
/api/paper-trading/sell/{trade_id}	POST	Close position
/api/paper-trading/trade/{trade_id}	PATCH	Edit SL / target
/api/paper-trading/reset	POST	Reset portfolio (?user_id=)

Infrastructure

Endpoint	Method	Description
/health	GET	Health check (keep-alive ping)

21. Frontend Pages & Components

Pages

Page	Route	Description
Landing	/ (unauthenticated)	Public marketing page — features, how-it-works, CTA adapts to login state
Dashboard	/dashboard	Movers (US/IN/Crypto), market status, live index bar with loading badge

Stock Detail	/stock/:symbol	Prediction, trade levels, factor breakdown, news, chart
Daily Picks	/picks	Top BUY ideas by horizon, portfolio weights, trust layer
Screener	/screener	Filter and explore the universe
Backtest	/backtest	Single-stock historical walk-forward test
Heatmap	/heatmap	Sector colour-coded snapshot (IN / US) with loading badge
Watchlist	/watchlist	Saved stocks with live prices and change% — user-scoped
Alerts	/alerts	Price alerts with live trigger detection — user-scoped
Portfolio	/portfolio	Holdings with BUY/HOLD/SELL per position
Paper Trade	/paper-trading	Simulated trading — open/close positions, P&L — user-scoped
Validation	/validation	Hit rate, Sharpe, alpha vs benchmark

Key Components

Component	Description
FactorAttributionWaterfall	Score decomposition bar chart; click to drill down
ConfidenceMeter	Colour-coded progress bar (0–100)
ConfidenceBreakdown	SVG gauge + 5 confidence bars with tooltips
BullBearCase	Analyst-style bull/bear thesis bullets
TradingViewWidget	Embedded chart (visual only; not wired to engine)
IndexBar	NIFTY 50, SENSEX, VIX live strip
SignalBadge	BUY / HOLD / SELL badge with colour
ScoreHistoryChart	Composite score trend over time
NewsCard	Sentiment-tagged news card
BacktestPanel	Walk-forward results per horizon on Picks page
LivePerformanceTracker	Per-pick P&L — entry, return%, alpha vs Nifty
PaperTradeModal	Trade form (qty, horizon, pre-filled SL/target)

Daily Picks Trust Layer

The Picks page has a collapsible **"Show Real Accuracy"** panel with three layers:

- 1. **Backtest results** — real walk-forward hit rate, avg return, Sharpe, alpha vs Nifty per horizon
- 2. **Confidence calibration table** — empirical hit rate per score band (60–65, 65–70, 70–75, 75–80, 80–85, 85–91) so users can see if higher confidence = higher win rate
- 3. **Live P&L tracker** — every past daily pick with entry price, current return%, and alpha vs benchmark

Pick Card UI

- Rank badge (#1–#5)
- Score band label (STRONG BUY / BUY / HOLD) with colour
- Sector tag
- Top 3 signals inline (▲ BULLISH, ▼ BEARISH, → NEUTRAL) without needing to expand
- Compact market regime bar

22. Infrastructure & Deployment

Hosting

Layer	Provider	Plan
Backend API	Railway	Hobby (\$5/month, always-on, no cold starts)
Frontend	Vercel	Hobby
Database	PostgreSQL (Railway) or SQLite (local)	—
Auth	Supabase	Free tier

Migrated from Render to Railway (Session 5) — Railway Hobby plan eliminates the free-tier cold-start problem (30-second spin-up delays) and provides a persistent, always-on server.

Environment Variables

Variable	Description	Default
ENVIRONMENT	production / development	development
DATABASE_URL	PostgreSQL connection string	SQLite fallback
USE_POSTGRES	1 = Postgres, 0 = SQLite	0
PICKS_SECRET	Secret header for /api/picks/generate	Required in prod
PICKS_UNIVERSE_LIMIT	Cap stock count for picks run	25
PICKS_CANDIDATES	Top N from Phase-0 momentum screen for deep prediction	751 (set in Railway)

SCREEN_BATCH_SIZE	NSE bulk download batch size (memory safety)	300
MIN_MCAP_CR	Min market cap in ₹ Cr for NSE universe	100
SCREENER_EMAIL	screener.in login (Indian fundamentals)	Required
SCREENER_PASSWORD	screener.in password	Required
FRONTEND_URL	Vercel frontend URL for CORS	Must be set in prod

Backend Startup Sequence

1. Unicorn starts FastAPI app
2. CORS middleware enabled (frontend HTTPS origin)
3. **Postgres schema initialised** (`init_db()`) — creates all tables if not exist
4. **screener.in login** — authenticated session established on boot (not lazily)
5. Prediction router with daemon-thread async pattern initialised
6. Universe refresh (async, 30s delay)
7. Keepalive loop (14-minute self-ping interval)
8. Outcome resolver (6-hour interval)
9. Warmup loop — pre-computes RELIANCE:IN:medium and AAPL:US:medium (150s delay, 90s gap)

23. Automation Workflows

Directory: `.github/workflows/`

daily_picks.yml — Daily Picks Generation

Schedule: `0 3 * * 1-5` (3:00 AM UTC = 8:30 AM IST, Mon–Fri) Step 1: Poll `/health` up to 10× (30s gap) until Render responds 200 — prevents silent failure when server is cold-starting Step 2: POST `/api/picks/generate` with `x-secret` header Purpose: Ensures picks are generated even after overnight sleep Result: Picks ready by 9:00–9:10 AM IST

weekly_validation.yml — Model Accuracy Validation

Schedule: `0 2 * * 0` (2:00 AM UTC = 7:30 AM IST, Sunday) Steps: 1. Ping `/health` to wake Render 2. POST `/api/validation/run?horizon=medium` → wait 15 min 3. POST `/api/validation/run?horizon=long` → wait 25 min 4. GET `/api/validation/results` → print summary Output: BUY hit rate %, alpha %, Sharpe per horizon

keep_alive.yml — Render Free Tier Keep-Alive

Schedule: `*/10 * * * *` (every 10 minutes, 24/7) Action: GET `/health` Purpose: Prevent Render free tier from spinning down

24. Persistence & Data Durability

Render's free tier uses ephemeral disk — files written locally are wiped on every restart/redeploy. All user-facing and learning data is stored in PostgreSQL to survive this.

What Lives in Postgres

Data	Table	Survives Restart
Price alerts	<code>price_alerts</code>	■ Yes
Watchlist	<code>watchlist</code>	■ Yes
Paper trades & portfolio	<code>paper_trades</code> , <code>paper_portfolio</code>	■ Yes
Daily picks cache	<code>daily_picks_cache</code>	■ Yes
Validation results	<code>val_runs</code> , <code>val_signals</code>	■ Yes
Alpha engine predictions	<code>predictions</code>	■ Yes
Outcome resolution	<code>outcomes</code>	■ Yes
IC history	<code>factor_ic_history</code>	■ Yes
Regime log	<code>regime_log</code>	■ Yes
Score snapshots	<code>score_snapshots</code>	■ Yes

What Is Transient (acceptable)

Data	Storage	Why Acceptable
Trained ML models (<code>meta_model_*.pkl</code> , <code>regime_kmeans.pkl</code>)	Local file	Auto-retrains from Postgres on next run — one cycle of degraded weights, no user data lost
API response caches (quotes, heatmap, movers)	In-memory (TTL)	Market data; freshly fetched anyway

screener.in Session

- Login fires at Render boot (not lazily on first request)
- Session refreshed every 6 hours
- `SCREENER_EMAIL` + `SCREENER_PASSWORD` must be set as Render environment variables
- Login logs: `[startup] screener.in login succeeded/failed` — check Render logs after deploy

25. Factor Weights by Horizon

Short-Term (1–5 days)

Factor	Weight	Rationale
Technical	70%	Momentum and price action dominate short windows
Fundamental	15%	Slow-moving signal, limited short-term relevance
Sentiment	15%	News-driven price moves matter in 1–5 days
Quality	Not applied	Computation overhead; not predictive short-term

Medium-Term (2–4 weeks)

Factor	Weight	Rationale
Technical	40%	Trend still matters but less dominant
Fundamental	45%	Growth and valuation start to drive returns
Sentiment	15%	News flow still relevant over multi-week horizon
Quality	Included in composite	Institutional signals begin to matter

Long-Term (3–6 months)

Factor	Weight	Rationale
Technical	15%	Mean reversion reduces technical edge
Fundamental	75%	Business quality and valuation dominate
Sentiment	10%	Structural, not ephemeral, news matters
Quality	Included, weighted heavily	Piotroski, ROIC, cashflow most predictive

26. Key Design Principles

No Look-Ahead Bias — Backtester uses only data available at the prediction date. Forward returns computed strictly after prediction timestamp.

Full Explainability — Every signal has reasoning bullets, factor breakdown, and confidence components. Nothing is a black box.

Honest Risk-Adjusted Returns — Target prices are not inflated. Trade levels (entry, stop, target) are mutually consistent with the signal.

Sector-Aware Scoring — Different valuation thresholds for banks (ROE > 15%, P/B < 1.5), NBFCs, and IT companies vs industrials.

Institutional-Grade Signals — Piotroski F-Score, Sharpe Ratio, ROIC, IC engine, Ledoit-Wolf covariance optimisation — same tools used by quant funds.

Data Resilience — Three-layer fallback chain: yfinance → screener.in → BSE API. If news is unavailable, weights redistribute gracefully.

Memory-Efficient — Designed for Render's 512 MB free tier. Cache capped at 300 entries with LRU eviction. Concurrent predictions use daemon threads, not async tasks.

Self-Improving — Outcome logger tracks every prediction. IC engine retrains weekly. Factor weights evolve as the model sees more real-world outcomes.

Real-Time Ready — 15-minute prediction cache, async background computation, React Query polling for live data.

Investor Transparency — Every number in the UI is traceable to a specific calculation in the codebase. This document is kept current with every code change.

Postgres-First Persistence — All user data (watchlist, alerts, paper trades, picks, validation, alpha engine) is stored in PostgreSQL. Render's ephemeral disk is never trusted for user-facing state.

27. Changelog

Session 9 — 2026-06-24

A very long session: resolved a real production infrastructure incident (duplicate Render+Railway deployments writing to the same database), fixed four separate event-loop-blocking bugs causing the slowness reports across Portfolio/Paper Trade/Heatmap/Crypto, closed three "signal contradicts its own data" gaps in the prediction and Multibagger engines, fully separated the Learning Alpha Engine by market (IN and US had never actually trained independently despite Daily Picks ranking both), and closed two critical Supabase security findings (Row-Level Security disabled on every table).

Infrastructure Incident — Duplicate Render + Railway Deployments:

- Render kept sending "exceeded memory limit" / repeated restart emails for a service the team believed was already fully migrated to Railway. Investigation found Render's `stocksense-api` service was never actually decommissioned — confirmed via its own logs that it was independently re-running the full Daily Picks generation pipeline and background prediction threads, completely unrelated to the Railway deployment serving real traffic.
- **Root cause, fully traced:** `_catchup_picks()` in `api/main.py`'s startup `lifespan()` runs on every process boot — if today's picks don't exist yet in Postgres, it generates them. Render and Railway share the *same* Supabase Postgres database (confirmed: identical connection string on both dashboards). Every OOM-triggered restart on Render re-ran this startup catchup; if the timing raced ahead of Railway's legitimate scheduled run, it kicked off a second full generation pass, consuming enough memory to OOM again — a genuine self-sustaining crash loop, not external traffic.
- Confirmed via the Railway Network tab that the live frontend's `NEXT_PUBLIC_API_URL` was already correctly pointed at Railway — the Render service had zero legitimate purpose left. Suspended (later safe to fully delete) via the Render dashboard; verified the live site continued working normally with zero behavior change.
- Side-effect noticed during the investigation: `postgres_store.py`'s connection pool doesn't set `prepare_threshold=None` like every other direct-connection call site in the codebase, which is why a `prepared`

statement `"_pg3_0"` already exists error showed up in the Render logs — Supabase's transaction-mode pooler (port 6543) doesn't reliably support psycopg's default prepared-statement caching across multiple concurrent client processes. Noted as a small follow-up; not separately fixed this session since the duplicate-deployment removal eliminates the only known case of two processes contending for the pool simultaneously.

Four Event-Loop-Blocking Bugs Found — the Real Cause of "Slow" Reports:

Methodically audited every `async def` in `backend/services` and `backend/api/routers` for one specific anti-pattern: a function declared `async def` that calls a *real* blocking network operation directly instead of through `loop.run_in_executor(...)`. A single such call freezes Python's one event loop for its *entire* duration — not just slowing the request that triggered it, but stalling every other concurrent request on the process, including completely unrelated ones (a bare `/health` check measured 13 seconds during one load test, simply queued behind an unrelated blocked call).

- **MarketDataService.get_quote()** (`market_data.py`) — the yfinance `fast_info` fallback/enrichment calls ran inline. This is what made Portfolio (one quote request per holding) and Paper Trading (one quote request per open trade, recurring every ~28s) feel slow as holdings/trades grew past a handful. Confirmed via direct load testing against production: 8 concurrent fresh quote requests took **15.4 seconds total** before the fix (all resolving within milliseconds of each other — the signature of serialized, not parallel, execution) versus **1.69 seconds** after.
- **MarketDataService.get_ohlcv()** — same pattern, used by the chart on every stock detail page.
- **crypto_engine.predict_crypto()** — the worst case: a rate-limit retry loop with `time.sleep()` directly inline, meaning a single throttled crypto request could freeze the *entire backend* for several seconds per retry attempt, compounding across attempts.
- **news_sentiment.NewsSentimentService.get_macro_news()** — called several sequential blocking `feedparser.parse()` RSS fetches directly; the per-stock news path was already correctly wrapped, only the macro feed (`/api/news/macro/us`, `/api/news/macro/india`) had this bug.
- All four fixed identically: moved the blocking call into a plain sync helper method, dispatched via `loop.run_in_executor(None, ...)` with a timeout, matching the pattern already used correctly elsewhere in each file.
- **Load-tested the fix directly against production** afterward: 100 concurrent fresh quote requests succeed 100% of the time (with growing but acceptable latency, up to ~12-17s in the worst case); the real ceiling where requests start failing (false "symbol not found" errors from all three data-source fallbacks getting saturated at once — not a crash) is somewhere between 100 and 150 concurrent fresh requests. Re-verified by re-querying one of the "failed" symbols in isolation immediately after — it succeeded instantly, confirming the failures were transient saturation, not a real bug.

Three "Signal Contradicts Its Own Data" Gaps Closed:

A user-flagged real paper trade (MU, +82% confidence BUY that dropped 13.18% the next session) led to tracing exactly why a high-conviction signal could still produce a bad outcome. The composite score and confidence weren't wrong — strong fundamentals genuinely supported the call, and the model *did* flag the risk that materialized ("**High beta (2.17) – amplified downside in market corrections**") — but that risk only cost a small score penalty, never enough to flip a confident-looking badge. Three concrete instances of this same root pattern were found and fixed:

1. **Unfavorable risk/reward never demoted confidence.** `_trade_levels()` already computes a real `risk_reward_ratio` (MU's was 0.36 — risking \$146.65/share to make \$52.59/share) but never fed it back into the signal. Added `_apply_risk_reward_adjustment()` in `prediction_engine.py`: when risk exceeds reward for a BUY/SELL signal, confidence is capped at 30 (pushing the badge into the gray/weak tier) and an explicit plain-language warning is added to both the reasoning list and bear case. Composite score and BUY/HOLD/SELL classification are deliberately left untouched, so Daily Picks ranking and the IC learning engine — which key off composite score, not confidence — are unaffected.

2. **Severe promoter pledge (IN) had the identical gap.** Pledge over 50% is already described in the fundamental-score reasoning as "*severe margin call risk; avoid,*" but that text only cost -8 points inside a ± 10 -capped governance bucket — easily absorbed by strong fundamentals elsewhere. Added `_apply_pledge_adjustment()`, same demotion-to-30 treatment, fires only for IN market + BUY signal + pledge > 50%.

3. **Multibagger's shortlist ranking ignored its own red-flag verdict.** `annotate_and_rank()` ranked the "Shortlisted" (top ~20%) tier purely by raw checklist score, completely ignoring `verdict/red` flags — reproduced with a synthetic case where a stock with 5 separate red flags and an explicit `verdict: avoid` still got `shortlisted: true`, tied on score with (and ranked ahead of, due to Python's stable sort preserving list order on ties) clean stocks with zero red flags. The frontend renders the shortlist flame icon and verdict badge independently per row, so this could show "Shortlisted" directly next to "Avoid" on the same row. Fixed: red flag count is now a secondary sort key (clean stocks win ties), and shortlist eligibility excludes any "avoid"-verdict stock outright regardless of raw score.

Daily Picks: One Dead Feature, One Gate Gap:

- `_predict_stock()` read `trade.get("risk_reward")`, but `_trade_levels()` returns the field as `risk_reward_ratio` — the key never existed. The "R:R 1:X" badge on every single Daily Picks card has been silently dead since it was built (the `&&` short-circuits on `undefined`); the stock detail page was unaffected since it reads the correct key. Fixed the key name.
- The picks-specific quality gate only required `confidence >= 25%` — but the two confidence-demotion fixes above demote to exactly 30, which clears that floor. A stock explicitly flagged with bad risk/reward or severe pledge could still land in the curated "Top 6" list. Added an explicit check excluding any pick whose reasoning carries a "Risk/Reward" or "Governance Risk" indicator, regardless of confidence — a curated best-ideas list should hold a higher bar than "not pure noise."

"Unknown" Sector Badge — Real Root Cause Found, Not Just Patched Again:

- The sector badge intermittently showing literal "Unknown" text (previously partially addressed in Session 8 area, see task #8 history) turned out to have two separate causes, both fixed this time: (1) the final fallback path in `sector_strength_score()` hardcoded the literal string "Unknown" instead of `None` when no sector text was available at all — even though the frontend already correctly hides the badge on a falsy value, "Unknown" is a non-empty string, so it always rendered; (2) the live screener.in scrape that's supposed to supply the fallback text is cached in-memory for 4 hours **including failures** — a single rate-limited request during the nightly ~150-call batch poisons that symbol's result for the rest of the window, and a *different* subset of stocks hits this each night, matching the "fixed once, reappearing for different stocks" report.
- Fixed the literal-string issue directly (return `None`, let the existing frontend conditional hide the badge). For the underlying data gap, added a second fallback layer: `stock_fundamentals_cache` (the Postgres table already maintained nightly for the Multibagger screen, persists across restarts) is now checked before giving up — a stock's sector classification doesn't change day to day, so once it's been successfully resolved once by the nightly refresh job, today's live scrape failing no longer matters.

Learning Alpha Engine — IN and US Were Never Actually Separate:

A deeper audit (explicitly requested after confirming the above fixes) found the entire IC engine / meta-model / outcome-resolution pipeline had no concept of market at all, despite Daily Picks ranking both IN and US stocks with it.

- `predictions/outcomes` tables had no `market` column. `outcome_logger._fetch_return` hardcoded the NSE `.NS` ticker suffix unconditionally — for a US stock this builds "AAPL.NS", which doesn't exist on Yahoo Finance, so every US outcome resolution silently failed forever. US predictions were logging fine; their outcomes could never resolve.
- **Net effect:** the IC engine's live IC and the meta-model were trained only on the IN outcomes that ever resolved, yet that single learned weighting was applied to rank both markets — US Daily Picks were effectively being ranked

by India-calibrated factor weights, never validated against a single real US outcome. This is also exactly what the "AI Engine: Learning..." badge on the Daily Picks page reflects (`meta_model: any(r.get("meta_alpha") is not None for r in top_buy)`) — US could never progress past "Learning..." no matter how long the app ran, since it could never accumulate the 100 resolved outcome pairs needed to train.

- Fixed end-to-end: added `market` column (default 'IN') to both Postgres and SQLite schemas, with a guarded one-time constraint migration (the old `UNIQUE(symbol, horizon, pred_date)` had to widen to include `market`, or an IN and US outcome for the same symbol/date would collide). Every function in the chain (`log_prediction`, `log_outcome`, `get_training_data`, `get_unresolved_predictions`, `count_training_rows`) now takes and respects `market`, on both store backends. `outcome_logger` resolves both markets with the correct ticker suffix per market.
- `ic_engine.ACADEMIC_PRIOR_IC` now has separate prior tables per market — IN unchanged (Nifty research), added distinct US priors sourced from real published research (Jegadeesh-Titman momentum, Fama-French 5-factor, AQR Quality-Minus-Junk), since US large-caps absorb sentiment faster and have different analyst-coverage dynamics than IN mid/small-caps. Also fixed a latent NaN-propagation bug found along the way: a zero-variance factor sample now falls back to the academic prior instead of poisoning the weight with NaN.
- `meta_model` now trains and persists a separate pickle per (market, horizon) instead of one shared model per horizon. `weight_adapter.run_adaptation(market)` retrains one market's IC cache + meta-models per cycle; Daily Picks calls it once per market in its own background thread. Regime KMeans retraining stays shared across markets by design — it's a genuinely global macro signal (VIX, DXY, US10Y, both Nifty and S&P trend), not a per-market one.
- Verified directly: IC weights and raw IC values are now provably distinct between IN and US with zero live data (correctly falling back to each market's own academic prior).
- **Known follow-up, out of scope this session:** `score_snapshots` (backs the per-stock History tab chart, not the learning/ranking loop) still isn't market-tagged — low risk given current symbol universes don't collide between markets, but worth revisiting if it ever causes a visible chart bug. Also flagged but not fixed: trained meta-model `.pkl` files are saved to local disk, which Railway/Render wipes on every restart — the exact same problem `postgres_store.py`'s own docstring already documents for the old SQLite setup, meaning trained models are lost on every redeploy and only return once enough new outcomes re-accumulate.

Critical Security Finding — Row-Level Security Disabled on Every Table:

- Supabase's automated Security Advisor flagged two critical issues: a publicly-accessible table with RLS disabled, and sensitive columns exposed without access restriction. Verified by full inventory across every schema-creation site in the codebase (`postgres_store.py`, `fundamentals_cache.py`, `alerts.py`, `portfolio.py`, `validation_engine.py`) — **all 18 tables** had RLS disabled, including ones with real personal data: `terms_acceptance` (name, mobile, IP address), `paper_portfolio` and `price_alerts` (email), `watchlist/portfolio_holdings/signal_feedback/nps_responses` (user IDs). Anyone with the project's URL and anon key — normally embedded in frontend JavaScript by Supabase's own design — could read, edit, or delete any of this directly through the auto-generated PostgREST API, completely bypassing the FastAPI backend's own access control.
- Fixed by adding `ALTER TABLE ... ENABLE ROW LEVEL SECURITY (idempotent)` to every table-creation site. No policies needed: this backend connects as the `postgres` role via Supabase's pooler (`postgres.<project-ref>username` format), which has `BYPASSRLS` by default in every Supabase project specifically so enabling RLS for the public REST API doesn't affect direct/admin access. Verified live post-deploy with a full write→read→delete cycle — zero behavior change for the app, public REST hole closed.
- **Process gap found and fixed along the way:** `validation_engine.py`'s schema init (`_init_db()`) only ran lazily on first call to a handful of functions, never from `main.py`'s startup `lifespan()` like every other table. This meant the RLS fix for `val_runs/val_signals` sat deployed but inert in production until something happened to call `/api/validation/results` — confirmed live via Supabase's Security Advisor still showing those two tables as vulnerable after the rest had cleared. Exported a public `init_db()` wrapper and added it to the startup `lifespan` so future schema changes to that file take effect on deploy, not on incidental first use.

- One remaining item from the same Advisor scan is **not** code-fixable: "Leaked Password Protection" (rejects known-breached passwords via HaveIBeenPwned at signup) is gated behind Supabase's Pro plan — confirmed by attempting to enable it on the dashboard and receiving an explicit plan-tier error. Documented as an accepted low-severity gap rather than upgrading purely for this one feature.

Smaller UI Fixes:

- US Daily Picks displayed its generation time in ET ("8:30 AM ET") while IN showed IST — converted to the actual cron trigger's IST equivalent ("6:00 PM IST", derived from the fixed 12:30 UTC GitHub Actions schedule, not affected by US daylight saving) so both markets show one consistent timezone.
- Switching the IN/US toggle on Daily Picks visibly shifted the header's button cluster — root cause was a missing `placeholderData: keepPreviousData` on the page's query, so data briefly went undefined mid-switch, collapsing the "Updated X ago" badge and regime panel. Fixed, and proactively applied the same fix to Multibagger's two queries since it has the identical toggle pattern.
- Multibagger Screen's IN/US toggle was moved above the title per one round of feedback, then reverted back onto the same line as the title (matching Daily Picks/Heatmap/Screener's standard layout) per a follow-up correction.
- Documented the Signal badge color grading (BUY's three confidence-graded shades, HOLD vs. REJECTED distinction, the dash meaning "queued in a staggered batch" not "no signal") in a new §18a, plus a glossary for Invested/Unr. P&L/Near stop loss/Mkt Price (last close when closed)/Composite Score/Multibagger Verdict-Shortlisted/REJECTED reasons — added after a user asked how to read a screenshot of mixed badge colors and confirmed there was no existing reference for it.

Session 8 — 2026-06-23

A long session covering a surgical bug-hunt pass across the multi-market changes, a thorough mobile/desktop CSS layout audit, and a real architecture fix (Portfolio finally syncing across devices). Documentation batching was changed mid-session from "after every push" to "once per day" per explicit founder feedback — this entry covers everything from that point through end of session in one pass instead of ~22 separate changelog entries.

Multi-Market Scoring Bugs Found and Fixed:

- **Sector/relative-strength scoring was India-only for every US stock.** `sector_strength_score()` looked symbols up in a static curated India-only map, so any US symbol always returned "Unknown" with a neutral score. Separately, `relative_strength_score()` always compared against Nifty 50 regardless of market — a US stock's "relative strength" was being computed against a rupee-denominated index. Fixed: US sector now comes from yfinance's own `GICS.info["sector"]` field (works for any US stock, no curated list needed) compared against SPDR Select Sector ETFs vs. the S&P 500; IN behavior is byte-for-byte unchanged. Bonus: the NSE Finance sector index ticker (^CNXFINANCE) was found delisted/404ing on yfinance — swapped to `NIFTY_FIN_SERVICE.NS`.
- **US Top Gainers/Losers showed far fewer than 10 names each.** The live-request fallback scanned ~50 of the 340+ curated US large-cap symbols one at a time via Finnhub (free tier, no bulk endpoint, hard timeout budget) — ~85% of the universe was never checked. Fixed by adding a periodic background job (`_us_movers_refresh_loop`, every 3 min) that scans the full universe via one bulk `yf.download()` call (~30s, fine for a background job, too slow for a live request) and pre-warms the cache.
- **Invite-link handling had two more gaps** beyond the earlier fix: it only processed hashes with `type=invite/type=recovery` exactly (re-inviting an email with an existing partial `auth.users` row can produce a different `type=`), and a reused/expired link (`#error=...` instead of tokens) gave zero indication anything was wrong. Both fixed — any hash with valid tokens is now processed regardless of type, and an error hash redirects to `/login?notice=invite_expired` with a clear message.

Alerts System Hardened to Match Paper Trading's Standard:

- **Server-side enforcement added.** Alerts previously only fired client-side via a 5s poll — closing the tab silently stopped monitoring with no backstop. New `services/price_alert_notifier.py` + `_price_alerts_check_loop`

(every 90s) scans non-triggered alerts with an email on file and notifies via the same Resend account used for invites/paper-trade alerts. Kill switch: `PRICE_ALERTS_ENFORCEMENT=0` env var disables just this background check without a code change.

- **Resurrection race fixed** — delete/reset fired the API call and updated local UI state unconditionally; a failed network call left the row alive in Postgres while the screen showed it gone, and the next page load's GET would silently bring it back. Now awaits the request first, only committing the local change on success (same bug class found and fixed independently in Portfolio later this session).
- **Screener filter unit mismatch + silent failures:** `min_roe` was compared against yfinance's raw ROE fraction while the output field is `*100`-scaled (not wired to the frontend yet, but a landmine); bare except: `pass` replaced with logged skip counts.

Surgical CSS/Layout Audit (mobile + desktop):

- **Stock detail page header** didn't stack on mobile — the AI signal panel's 140px minimum width left almost no room for price/badges on a ~375px phone. Now stacks vertically below `sm:`.
- **Two tables (Watchlist, Alerts) were missing `overflow-x-auto`** wrappers that every other table in the app already had — verified all `<table>` usages app-wide to confirm these were the only gaps.
- **Daily Picks and Dashboard header rows overflowed past the screen edge** on narrower viewports — both had an outer row that correctly wrapped, but an *inner* row (toggle + buttons + status badge) with no `flex-wrap/overflow-x-auto` of its own, so 3+ children forced onto one unbreakable line wider than the viewport. Fixed both, plus added a second safety layer (`flex-wrap` on the parent too) since in some browsers a child's `overflow-x-auto` doesn't get factored into the parent's own wrap decision.
- **Desktop navbar market-status block overflowed at laptop widths.** `MarketStatusInline` (NSE/NYSE/Crypto status + "Opens/Closes at..." text) was `shrink-0` with no width cap — US/Crypto's longer status text pushed the whole navbar row past the viewport right at the `lg:` breakpoint where it first appears. Since this lives in the shared root layout, it affected every page at once. Capped to `max-w-[42vw]` with its own internal scroll.
- **A genuine regression, self-corrected:** added `overflow-x: hidden` on `html/body` as a "safety net" against page-wide horizontal scroll — this didn't fix anything, it just removed the user's ability to scroll right to reach content that was still overflowing (worse, not better). Reverted, then found and fixed the actual overflow sources instead.
- **Missing header icons:** Market Overview, Market Heatmap, and Stock Screener were the only 3 of 9 main pages with no icon before the title (every other page already had one) — added `LayoutDashboard`, `Flame`, `Filter` respectively.
- **Market toggle styling unified** across Daily Picks, Dashboard, Heatmap, and Paper Trading (previously: compact segmented pills on some pages, larger individually-bordered pills with full country names on others) — all now use the same compact segmented-control style. Also standardized the three form-field market selectors (Alerts, Portfolio, Backtest).
- **Unresolved, flagged not fixed:** a navbar avatar/search-bar overlap reported on both Android and iOS, in what looks like a constrained in-app webview (Safari View Controller / Chrome Custom Tabs) rather than the full browser app — ruled out backdrop-blur compositing, global CSS, viewport meta conflicts, and the flexbox math itself (which cannot produce real box overlap as written) across multiple passes. Locked `maximumScale: 1` on the viewport as a defensive measure. Asked the founder to confirm whether it reproduces in the full standalone browser before further investigation, since computer-use/browser-automation tools were unavailable in this environment to verify directly.

Predictive Search Added to Portfolio and Alerts:

- Both pages had a bare text `<input>` for entering a symbol — no autocomplete, unlike the global search bar and Watchlist's add-stock field. Extracted the universe-loading + local-matching logic (previously duplicated between `SearchBar.tsx` and `watchlist/page.tsx`) into a shared `useStockSearch` hook + `StockSymbolField` component; picking a suggestion also auto-syncs the page's IN/US toggle to the selected stock's actual market.

- **Known gap, deferred to next session:** the autocomplete only matches against a static curated `stock_universe.json` (~1,585 US tickers), not the full market — confirmed RKT (Rocket Companies) and UUUU (Energy Fuels) are missing. Manual entry + Enter still works as a fallback even without a match.

Market Selection Now Persists:

- Every page with a market toggle (Daily Picks, Dashboard, Screener, Backtest, Alerts, Portfolio, Heatmap, Paper Trading) reset to "IN" on every refresh, with no memory of the user's choice. New `useMarketPreference` hook persists to one shared `localStorage` key across all 8 pages — picking "US" on one page carries the preference to the others, not just the same page on reload. Switched the internal read from `useEffect` to `useLayoutEffect` to eliminate a visible flash of the wrong market before the stored value applied.

Watchlist Split by Market:

- Was rendering all stocks (IN + US mixed) in one combined table with a Market column — inconsistent with Portfolio and Paper Trading, which both already group by market. Split into separate "■■ Indian Stocks" / "■■ US Stocks" sections, same pattern as the other two pages.

Portfolio: Inline Edit + Cross-Device Sync (the big one):

- **Inline edit added** for Qty and Avg Buy — previously the only way to fix a typo'd value was deleting and re-adding the holding. Pencil icon toggles to editable inputs, confirmed with a checkmark/Enter or cancelled with X/Escape (same pattern as Paper Trading's open-positions table).
- **Root architecture gap found and fixed:** Portfolio was the one feature still stored entirely in the browser's `localStorage` — Watchlist, Alerts, and Paper Trading all already synced per-user through Postgres, but a holding added on desktop was invisible on any other device for the same account, since nothing was ever sent to a server. New `portfolio_holdings` table + full CRUD endpoints (`backend/api/routers/portfolio.py`), mirroring the `ensure_table()`-on-startup pattern already used by Alerts. `localStorage` is now just a fast-access cache; on first load after this shipped, any holdings already sitting in local storage get migrated up to the server automatically so existing users don't lose them. Delete/edit also await the backend call before updating local state, same fix as the Alerts resurrection bug above.

Session 7 — 2026-06-22

Dashboard Tab Renamed "Gold & Silver" → "Commodities":

- More professional label, and leaves room to add more commodity instruments later (e.g. oil, platinum) without relabeling the tab again. `COMMODITY` internal key/type was already generic — only the two display strings (tab label, section heading) needed updating.

Crypto Fundamentals Check + Dead-End Tab Removal:

- Verified crypto's actual signal engine (`crypto_engine.py`) already correctly skips fundamentals entirely — technical + fear/greed (volatility proxy) + on-chain proxy (volume-based) only, explicitly documented as "no fundamentals." (An initial direct test of `PredictionEngine.predict()` for a CRYPTO market was misleading — that's dead code for crypto in production; `predictions.py`'s router actually dispatches CRYPTO to `predict_crypto()` instead.)
- The real issue was UI, not logic: the **Fundamentals tab** (`screener.in` data) rendered unconditionally on every stock detail page, but `screener.in` only covers Indian companies — for US, Crypto, and the tracking-only commodity ETFs it could only ever show "available for Indian (NSE) stocks only." Filtered it out of `HORIZON_TABS` for any market other than IN, since there's no scenario where it shows real data otherwise.

Gold & Silver Dashboard Tab:

- GLD/SLV/GOLDBEES/SILVERBEES were only reachable via search before this — no discoverable spot on the app's main landing page. Added a 4th market tab on the Dashboard (`frontend/src/app/dashboard/page.tsx`) next to India/USA/Crypto, showing live price + day change for all four as simple cards, same pattern as the existing Crypto tab's grid.
- No index bar (no natural index for 2 commodities — same simplification already made for Crypto) and no Quick Access/horizon-info sections (not meaningful for a fixed 4-symbol list) — just the price cards plus a note that these are tracking-only with no AI signal.

Gold/Silver Tracking Support — Fixed Fabricated Analysis:

- **Caught live:** after adding GLD/SLV/GOLDBEES/SILVERBEES to the universe, visiting their stock detail page produced a confident SELL signal whose bear case claimed *"Underperforming the Nifty 50 benchmark"* for a US gold ETF, plus invented valuation/earnings claims — despite `confidence_breakdown.data_completeness` being 0. The fundamentals/sentiment/quality pipeline was fabricating plausible-sounding analysis from data that didn't exist.
- **Fixed in `PredictionEngine.predict()`:** added an early-return branch for `TRACKING_ONLY_SYMBOLS` (`stock_universe.py`) that computes the signal from real technical indicators only (RSI/MACD/EMA/ADX/volume — legitimate for any price series) and skips the entire fundamentals/sentiment/quality/bull-bear-case pipeline, returning a `tracking_only: true` flag and an honest explanatory note instead.
- **Frontend:** stock detail page now shows a clear "price-tracking instrument" banner and hides Bull/Bear Case, Factor Attribution, and Academic Quality Signals when `tracking_only` is set.
- **Bonus fix:** this surfaced two latent null-pointer crashes (`prediction.fundamental_score.score / sentiment_score.score` accessed without a null guard) that would have thrown for *any* symbol with missing fundamental data, not just the new commodity ETFs — fixed with optional chaining.
- Added GLD/SLV (US) and GOLDBEES/SILVERBEES (India) ETF tickers to both `backend/services/stock_universe.py` and `frontend/public/stock_universe.json`, so they're searchable and usable in Watchlist, Alerts, and the stock detail page (price + chart) via existing infra.
- **Deliberately scoped to tracking only** — no AI BUY/SELL signal, no entry in Daily Picks. Fundamental factors (P/E, ROE, Piotroski score) that drive equity scoring don't meaningfully apply to a commodity ETF; full signal support would need a separate technical+macro-only model, considered and explicitly deferred rather than built half-heartedly into the existing equity-scoring engine.
- Both lists are marked as manual additions outside the universe auto-generator's source (S&P/NASDAQ-100 + all NSE equities) — they'll need to be re-added if `scripts/generate_stock_universe.py` is ever re-run.

Full Indian Holiday Calendar + Muhurat Trading:

- **Gaps found:** the frontend's `marketHours.ts` was missing several *fixed-date* NSE holidays (Ambedkar Jayanti Apr 14, Maharashtra Day May 1, Christmas Dec 25, Good Friday) — it already had the Easter-algorithm machinery for US holidays but wasn't applying it to India. The lunar/regional holiday list (`NSE_EXTRA_HOLIDAYS`) was completely empty. The backend's market-hours check (duplicated in `paper_trading.py` and `scraper_service.py`) had **zero** holiday awareness — not even the fixed ones.
- **Fixed:** added the missing fixed holidays (including Good Friday via the existing Easter computation) to `nseMarketHolidays()`; populated `NSE_EXTRA_HOLIDAYS` with the verified 2026 list (Holi, Ram Navami, Mahavir Jayanti, Bakri Eid, Moharram, Ganesh Chaturthi, Dussehra, Diwali, Guru Nanak Jayanti) sourced from NSE's official circular via Zerodha's mirror; wired it into the `nextEvent()` lookahead too (previously ignored).
- **Muhurat trading** — the special ~1hr evening session NSE/BSE run on Diwali Laxmi Pujan despite that date otherwise being a holiday — is now modeled via a `MUHURAT_SESSIONS` override list. 2026 falls on Sunday Nov 8; exact timing is a **placeholder** until NSE publishes the official window (~2 weeks before Diwali) — needs a follow-up update closer to that date.
- **Backend/frontend sync:** extracted a shared `backend/services/market_hours.py` mirroring the frontend logic exactly (same Easter computation, same 2026 extra-holiday list) and pointed both `paper_trading.py` and

`screener_service.py` at it, removing two inline duplicates that were drifting out of sync with the frontend and with each other.

- **Operational note:** `NSE_EXTRA_HOLIDAYS` (both the TS and Python copies) needs a manual refresh every December for the following year, from NSE's official circular at nseindia.com/resources/exchange-communication-holidays.

Two Manuals Added:

- `StockSense360_Technical_Handbook.docx` — internal architecture/AI-engine/infra reference for engineers and the founder.
- `StockSense360_User_Guide.docx` — end-user/investor-facing walkthrough of every page, with 16 real screenshots captured from production (also doubles as a visual confirmation that this session's fixes shipped correctly).
- Both manuals later updated again in this session to reflect the market-hours gating, holiday calendar, Commodities tab, and tracking-only-instrument changes below (the manuals had drifted behind the markdown/PDF changelog).

US Market Support Added to Daily Picks:

- **Root question:** Daily Picks only ever covered NSE India — not a technical limitation, just never wired up. `PredictionEngine.predict()` already accepted a `market` parameter and worked correctly for US stocks (used on every US stock detail page); the daily-picks batch job (`daily_picks.py`) was the only piece hardcoded to `market="IN"`.
- **Backend:** generalized the bulk momentum screener (`_bulk_screen_nse` → `_bulk_screen(market, ...)`), the mcap-floor screener (NSE exchange code `NSI` vs US exchange codes `NMS/NYQ/NGM/ASE/PCX`), the regime-detection proxy ticker (`RELIANCE` → `AAPL` for US), the per-stock prediction call, and the disk/Postgres cache — all now take a `market` argument instead of assuming India. Added a 100-ticker US mega-cap fallback list (mirrors the existing Nifty 100 fallback role) for when the live screener call fails.
- **Postgres:** added a `market` column to `daily_picks_cache` (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS`) so `save_picks_to_db/load_picks_from_db` keep independent history per market instead of one market overwriting the other's cache row.
- **API:** `/api/picks/daily`, `/api/picks/status`, and `/api/picks/generate` all accept a `market=IN|US` query param (default `IN`, validated against an allow-list). The `_generating/_last_error` module state in `daily_picks.py` changed from a single bool/string to a dict keyed by `market`, so an `IN` run and a `US` run can't trip each other's "already running" guard.
- **Cron:** GitHub Actions workflow (`daily_picks.yml`) now runs two scheduled jobs — India at 20:30 UTC (2 AM IST, unchanged) and US at 12:30 UTC (~8:30 AM ET, comfortably before the 9:30 AM ET open) — each calling `/api/picks/generate` with its own `market` param.
- **Frontend:** Daily Picks page (`/picks`) gets an `IN/US` toggle next to the existing horizon tabs. Switching markets changes the query key (separate cache per market), currency symbol (₹/\$), number locale (`en-IN/en-US`), display timezone for "Updated at," and the market passed through to the stock-detail link and the Paper Trade modal — all previously hardcoded to India.
- **Known limitation, not fixed in this pass:** the live picks-performance tracker (`/api/picks/performance`) and score-snapshot history key only on (`symbol, horizon`), with no `market` column — if a US ticker and an NSE ticker ever share the same symbol, their historical performance rows could blend. Low real-world risk (ticker collisions across the two universes are rare) but worth a follow-up if it's ever seen in practice.

Separate USD Ledger for US Paper Trading:

- **Bug caught during the US Daily Picks review:** paper trading had exactly one `cash` column on `paper_portfolio`, denominated in ₹10,00,000, shared across every trade regardless of market. Buying a \$100 US stock would have deducted "100" from that same rupee pool — silently treating dollars as rupees, with no currency

conversion or separation at all.

- **Fixed:** added a `cash_usd` column (`ALTER TABLE paper_portfolio ADD COLUMN IF NOT EXISTS cash_usd DOUBLE PRECISION NOT NULL DEFAULT 10000.0` — \$10,000 starting balance, not a currency-converted equivalent of the ₹10,00,000 IN balance, just a separate round-number virtual pool). `paper_trading.py`'s buy/sell/edit/reset endpoints now resolve which cash column to debit/credit based on the trade's `market` field instead of always touching `cash`.
- **Reset endpoint** now takes an optional `market=IN|US|ALL` param (defaults to `ALL` for backward compatibility) so a user can wipe just one market's trades/cash without touching the other.
- **Frontend:** the Paper Trading page (`/paper-trading`) gets an IN/US toggle (same pattern as Daily Picks and Dashboard). Cash, invested amount, unrealized/realized P&L, and the reset confirmation are all scoped to the selected market — open/closed trades are filtered by market before any totals are summed, so dollars and rupees are never added together.
- **Follow-up same session:** bumped the US starting balance from \$10,000 to **\$100,000** per founder request, with a one-time Postgres backfill (`UPDATE paper_portfolio SET cash_usd = 100000.0 WHERE cash_usd = 10000.0`) so the handful of portfolios created in the brief window the old default was live also got the new amount.

US Daily Picks Catch-Up Recovery (matching India's):

- **Root question:** with two scheduled cron runs now (IN at 2 AM IST, US at ~8:30 AM ET), does a missed/crashed run recover on its own for both markets? It didn't — the existing catch-up safety net in `api/main.py` (`_catchup_picks`, fires on server startup, regenerates today's picks if the scheduled cron never completed) was IN-only.
- **Fixed:** generalized `_catchup_picks()` to take `(market, tz_offset_hours, trigger_hour, settle_secs)` instead of hardcoding IST/2 AM, and scheduled a second instance for US — checks after 9 AM (fixed UTC-5 offset, matching the same non-DST-aware simplification already used by `picks_generated_today("US")`) on weekdays, regenerates if no US picks exist for today yet.
- **Bonus fix:** the validation catch-up task (`catchup_task`) was missing from the FastAPI lifespan's shutdown cancellation list — a pre-existing leak unrelated to picks, fixed while touching the same block.

US Market-Hours DST Bug Fixed:

- **Found while explaining the IN/US holiday-handling comparison:** `backend/services/market_hours.py` computed Eastern Time using a hardcoded UTC-4 offset, with its own comment admitting *"approximation, ignores EST/EDT switch"* — while the frontend's `marketHours.ts` correctly used the IANA `America/New_York` zone (auto-adjusting for daylight saving via `Intl`). The two only agreed during EDT (roughly mid-Mar–early Nov); during EST months the backend's `is_market_open("US")` — which gates paper trading buy/sell orders — was off by an hour from the real market clock and from what the frontend displayed.
- **Fixed:** switched `market_hours.py`, `daily_picks.py`'s `picks_generated_today("US")`, and the US Daily Picks catch-up scheduler in `main.py` to `zoneinfo.ZoneInfo("America/New_York")` instead of a fixed offset — all three (and the frontend) now agree year-round. Verified directly: `ZoneInfo` correctly returns `-05:00` for a January date and `-04:00` for a July date.
- India's side was never affected — IST has no daylight-saving rule, so a fixed `+05:30` offset is always correct.

Market-Hours Gating for Paper Trading:

- **Root issue:** Buy/Sell could execute instantly at any time of day using a stale last-close quote when the market was closed — unrealistic (a real market can gap through that exact price by next open) and not a good look for a tool positioning itself as a serious research platform.
- **Frontend:** `PaperTradeModal` now checks `getMarketStatus(market)` (the same utility already driving the navbar's market-status pills), shows a `"{market} market is closed — opens at ..."` banner, and disables the submit button while closed.

- **Backend:** mirrored the same check in `paper_trading.py`'s `/buy` and `/sell` endpoints (a local `_is_market_open()`, same pattern already used in `screeener_service.py`) — a direct API call would otherwise bypass the frontend-only gate entirely.

Paper Trade Target/Stop-Loss Proximity Notifications:

- **New services/trade_notifier.py** — a background loop (every 15 min, registered in `main.py`) scans all OPEN paper trades with a `target_price` or `stop_loss` set, fetches the live quote, and emails the owner once price is within 2% of (or has crossed) either level. Each trigger is deduped via `target_notified_at/stop_notified_at` columns + a 6-hour cooldown, so a price hovering near the line doesn't spam the same email repeatedly.
- **Sends via Resend's HTTP API directly** (not through Supabase's SMTP) from `alerts@stocksense360.com` — a distinct sender from `invites@` so users can tell notification types apart. Requires a `RESEND_API_KEY` env var on the Railway backend service (reuses the same Resend account/API key set up for invite emails).
- **paper_portfolio.email** column stores the user's email captured from the frontend (`useAuth().user.email`) on every `/buy` and `/portfolio` call — no Supabase admin API call needed on the backend.
- **Browser popup notifications** — `OpenTradeRow` fires a `Notification()` once per (trade, kind) per session when price nears target/stop, gated on `Notification.permission === "granted"`. New "Enable Notifications" button added to the Paper Trading page header (browser permission prompts require a user gesture).

Daily Picks Generation Crash Fix:

- **Root cause found** — the 2 AM IST daily picks cron run crashed every single day on the short-term overbought-RSI quality gate in `daily_picks.py`: `" ".join(r.get("reasoning", []))` assumed reasoning was a list of plain strings, but it's actually a list of structured dicts (`{"indicator":..., "reason":...}`) built in `prediction_engine.py` for the factor-breakdown UI. The crash threw `TypeError: sequence item 0: expected str instance, dict found`, caught by the top-level crash handler, which silently saved an empty fallback payload (`{"short": [], "medium": [], "long": []}` + an error field) instead of real picks — so the Daily Picks page showed either a stale "Generating picks..." spinner (during catch-up retries) or "No BUY signals found today" (a misleading message; it wasn't that no signals existed, the run never got far enough to find any).
- **Fixed:** extract `item.get("reason", "")` from each dict before joining, with a defensive fallback for plain strings.
- **Verified live** via `/api/picks/status` — confirmed the crashed run's error field, then watched a fresh catch-up-triggered run complete with `last_error: null` after the fix deployed.

Daily Picks Target-Price / Upside Methodology (clarified, not a bug):

- Confirmed via `prediction_engine.py::_estimate_target()`: every **BUY** signal's target price has a hard-coded floor relative to current price — medium-horizon floors at `price * 1.05`, long-horizon at `price * 1.15`, short-horizon via an ATR-based move. This means **every BUY pick is guaranteed to show positive upside by design**, not because each stock's underlying analyst-target/trend math happened to be positive. If several BUY picks' natural projections fall below the floor, they'll cluster at exactly the same floored upside % (e.g. several stocks all showing "+5.0% upside" identically) — that's expected behavior, not a calculation bug, but worth knowing when interpreting the displayed upside numbers as "genuine" per-stock projections.

Validation Universe Bug — India Results Were Permanently Shadowed:

- **Root cause found** — `val_runs` had no `universe` column. `get_latest_results()` / `get_per_stock_results()` only filtered by horizon, picking whichever run had the highest id (i.e. most recent). Since the daily validation schedule always runs `nifty100` → `midcap` → `us` in that order, the **US run is always the most recent** for any given horizon — so the Validation page always displayed US results, and India (`nifty100/midcap`) results, though present in the database, were never shown.

- **Fixed:** added a `universe` column to `val_runs` (Postgres: idempotent `ALTER TABLE ADD COLUMN IF NOT EXISTS`, since the table already existed in production and `CREATE TABLE IF NOT EXISTS` alone is a no-op on existing tables; SQLite: `ALTER TABLE` wrapped to ignore "duplicate column" on repeat init). `get_latest_results()`/`get_per_stock_results()` now accept and filter by `universe` (default `nifty100`).
- **API:** `/api/validation/results`, `/results/stocks`, `/results/stock/{symbol}` all now accept a `universe` query param.
- **Frontend:** Validation page now has a Nifty 100 / Midcap / US selector above the horizon tabs. All "vs Nifty" benchmark text is now dynamic — shows "vs S&P 500" when viewing the US universe instead of incorrectly saying Nifty for US data.

Railway Redeploy Scoping:

- **Root cause found** — Railway redeployed the backend service on every push to `main`, regardless of which files changed, including pure frontend and documentation commits. Each restart re-runs the startup "catch-up" check in `main.py`, which can kick off a brand-new ~10-15 minute full picks-generation run if it lands in a window before the day's legitimate 2 AM IST cron run has finished persisting — producing duplicate/wasted runs and a confusing "Generating picks..." spinner shown against an already-complete day's data.
- **Fixed:** added `railway.json` at repo root with `build.watchPatterns: ["backend/**"]`, so the backend service only redeploys when backend code actually changes. Frontend and docs-only pushes no longer restart it.

Invite Registration Fix:

- **Root cause found** — invite links (and password-reset links) authenticated the user for one Supabase session via magic-link code exchange, then dropped them straight onto `/accept-terms`. The user never set a password. On their next visit, `/login` only offers email + password sign-in with no Sign Up option (by design — invite-only app) — so an invited user with no password had no way back in.
- **New `/auth/set-password` page** — shown right after the invite/reset link authenticates the session; lets the user create a real password (min 6 chars, confirm match) via `supabase.auth.updateUser({ password })`, then continues to `/accept-terms`. Shows a clear "link expired" message if there's no active session.
- **`/auth/callback/route.ts` updated** — now redirects to `/auth/set-password?next=/accept-terms` instead of straight to `/accept-terms`. This covers both the invite flow and the forgot-password flow (previously forgot-password also had no way to actually set the new password after clicking the reset link).
- **Login page footer clarified** — explains invited users should look for an invite email with a link rather than expecting a Sign Up form.
- **Operational note:** Supabase Auth → URL Configuration must have `<site-url>/auth/callback` in the allowed Redirect URLs list for invite/reset links to work at all. If invites still fail after this fix, check that setting in the Supabase dashboard.
- **Production domain config fixed in Supabase** — Site URL updated from the default `*.vercel.app` to `https://stocksense360.com`; Redirect URLs now include `stocksense360.com`, `stocksense360.in`, `www.stocksense360.in`, and the `vercel.app` fallback, each with `/auth/callback`.
- **Second root cause found** — Supabase's dashboard "Invite user" button (and password-reset emails) don't support a custom redirect target; they always send the user to the bare **Site URL root** using the older **implicit/hash-based** flow: `https://stocksense360.com/#access_token=...&type=invite`. Hash fragments never reach the server, so the server-side `/auth/callback/route.ts` (which only handles `?code=...`) never even saw these — the user landed authenticated-but-unnoticed on the public homepage with no path forward.
- **InviteHashRedirect added to `providers.tsx`** — runs on every page load app-wide; checks `window.location.hash` for `access_token + type=invite/type=recovery`.
- **Third root cause found** — `@supabase/ssr's createBrowserClient` (used in `lib/supabase.ts`) does **not** auto-detect/establish a session from hash-fragment tokens the way the classic `supabase-js` client does (no `detectSessionInUrl`). The first version of `InviteHashRedirect` only checked for the hash and redirected to `/auth/set-password` — it never actually called `setSession()`, so the page found no real session and would show

"link expired" even on a valid, unused invite token.

- **Fixed:** `InviteHashRedirect` now parses `access_token` + `refresh_token` out of the hash with `URLSearchParams` and calls `supabase.auth.setSession({ access_token, refresh_token })` before navigating to `/auth/set-password?next=/accept-terms`.
- **Operational note — invite tokens are single-use:** clicking an invite/reset link consumes the token at Supabase's `/verify` endpoint regardless of whether the app does anything useful with the result. Any invite clicked before this fix shipped needs a **fresh invite resent** — the same link won't work twice.

Branded Invite Emails (Custom SMTP):

- **Custom SMTP via Resend** configured in Supabase → Authentication → Emails → SMTP Settings, sending from `invites@stocksense360.com` instead of Supabase's shared default sender — fixes deliverability and removes the "looks like spam" concern with invite emails.
- **Domain verification:** `stocksense360.com` added to Resend with DKIM (TXT), MX, and SPF (TXT) records added in GoDaddy DNS. Domain-level status can lag behind individual record checkmarks — re-trigger verification from the Resend domain detail page if status shows "Not Started" despite green record checkmarks.
- **Branded HTML invite template** (`supabase_invite_email_template.html` in repo root) pasted into Supabase's "Invite user" email template — dark themed, StockSense360 logo, "Team StockSense360" sender framing (no individual name), clear one-time-link messaging.
- **Diagnosing SMTP failures:** Supabase's "Failed to invite user" toast doesn't show the real cause — query `auth_logs` in Logs → Explorer (`select cast(timestamp as datetime) as timestamp, event_message, metadata from auth_logs order by timestamp desc limit 10`) to see the actual GoTrue/SMTP error (e.g. domain not verified, bad credentials).

Per-User Terms Cookie Bug:

- **Root cause found** — the `ss_terms=v1.0` cookie set after accepting the Terms of Use disclaimer was **not scoped to a specific user** (`path=/` with a fixed name). Any user authenticating in a browser that had *previously* accepted terms as a *different* account would see the cookie, skip `/accept-terms` entirely, and land straight on `/dashboard` — never asked for name/mobile/country or shown the legal disclaimer.
- **Fixed in two places:** `accept-terms/page.tsx` (both the read-check and the write-after-accept) and `useAuthGuard.ts` (the app-wide route guard) now use a cookie scoped per user: `ss_terms_${user.id}=v1.0`. The guard fix matters more — it could previously let a user bypass the disclaimer-acceptance redirect entirely on a shared browser, not just skip the profile form.

Session 6 — 2026-06-20

User Feedback System:

- **Signal thumbs up/down** — users can rate each AI signal (BUY/HOLD/SELL) directly on the stock detail page. Widget renders below the Paper Trade button; shows current vote highlighted in bull/bear colour. Votes are upserted per (`user_id`, `symbol`, `market`, `horizon`) so toggling works cleanly.
- **Monthly NPS survey** — `NpsPopup` component appears globally (bottom-right) after a user has voted on at least one signal and then every 30 days thereafter. 0–10 score card + optional free-text comment. Colour-coded (green ≥ 9 , yellow 7–8, red ≤ 6).
- **New backend router** (`backend/api/routers/feedback.py`) with 4 endpoints:
 - `POST /api/feedback/signal` — upsert thumbs vote
 - `GET /api/feedback/signal/{symbol}` — fetch user's existing vote
 - `GET /api/feedback/signal/summary/{symbol}` — aggregate approval % across all users
 - `POST /api/feedback/nps` — submit NPS score + comment

- GET /api/feedback/nps/due — returns {due: bool} based on 30-day cadence
- **DB schema additions** (already in postgres_store.py SCHEMA_SQL):
- signal_feedback table: per-user signal votes with UNIQUE constraint and ON CONFLICT upsert
- nps_responses table: per-user NPS scores with timestamps for 30-day cadence check

Look-ahead Bias Fix (Validation Engine):

- _backtest_stock() now recomputes indicators on a rolling window (df.iloc[:i+1]) at each signal date instead of on the full historical DataFrame. This eliminates future-price leakage into EMA-200, MACD, and OBV at time t.
- MIN_WARMUP raised 50 → 200 bars to ensure EMA-200 is valid before scoring begins.
- US symbols detected via is_us flag (.NS suffix no longer blindly appended).
- Validation hit rates will be modestly lower but accurately reflect real-time model performance.

Validation Coverage Expansion:

- Added NSE_MIDCAP universe (100 non-Nifty-100 NSE stocks) to validation_engine.py.
- Added US_BASKET universe (48 S&P 500 stocks spanning all GICS sectors).
- Railway cron (_validation_schedule_loop in main.py) now cycles all 3 universes (nifty100, midcap, us) back-to-back with 5-minute gaps between each.
- Deleted daily_validation.yml GitHub Action (was double-running alongside Railway cron).
- /api/validation/run endpoint accepts universe query param (nifty100 | midcap | us).

Session 5 — 2026-06-20

User Identity & Persistence:

- **All features migrated to Supabase user_id** — watchlist, alerts, paper trading, and StockContextMenu all previously used a hardcoded USER_ID = "default" or a localStorage session_id. Every feature is now scoped to the authenticated Supabase user UUID (useAuth().user.id), making data persist correctly across all browsers and devices.
- **Paper trading backend rewritten** — all 5 API endpoints migrated from session_id (random localStorage UUID) to user_id. BuyRequest, SellRequest, EditRequest models updated; _ensure_portfolio() queries by user_id. Old trades with user_id = NULL are legacy-only.
- **Dashboard watchlist fixed** — was fetching /api/watchlist/default; now uses userId from useAuth.

Heatmap Expansion & Quality:

- **India: 18 → 25 sectors** — added Healthcare, Insurance, Chemicals, Cement, Metal & Mining, Defence, Realty, Telecom, Consumer Disc, Hotels & Travel, Food & Beverage, Media & Entmt, Textiles, Agro & Chemicals, Logistics, Paints, Infra, Capital Goods, Power, EV & New Energy
- **US: 13 → 29 sectors** — added Cybersecurity, Fintech, Biotech, Med Devices, Clean Energy, EV, Consumer Stap, E-commerce, Social Media, Streaming & Media, Gaming, Airlines, Cruise & Hotels, Restaurants, Retail, Telecom, Utilities, Realty, Materials, Crypto & Blockchain
- **MAX_STOCKS raised 10 → 15** — wider coverage per sector
- **Full symbol audit** — all 353 India heatmap symbols bulk-tested via Yahoo Finance. 37 bad symbols identified; confirmed replacements applied (ATGL, ASTERDM, BAYERCROP, CANFINHOME, DALBHARAT, GUJGASLTD, ICICIPRULI, KNRCON, LEMONTREE, MFSL, MTARTECH, NH, ORIENTCEM, TEJASNET, TIPSFILMS, VTL, VIJAYA, VINATORGA, WAAREEENER, WELSPUNLIV, ZENSARTECH, ETERNAL). Symbols with no Yahoo Finance listing removed (SPICEJET, GREENKO, HEXAWARE, KEYSTONE, etc.)
- **ZOMATO → ETERNAL** — company rebranded to Eternal on NSE

- **NSE_SECTOR_TO_NSE_INDEX expanded** — 13 new sector → NSE index mappings added for primary data sourcing via NSE APIs

UX Fixes:

- **Landing page auth confusion fixed** — page now detects login state via `useAuth`; CTA switches from "Sign In" to "Go to Dashboard" when logged in; bottom CTA shows user email
- **Loading status badge** added to Market Overview (dashboard) and Heatmap — three states: blue "Fetching..." (first load), yellow "Refreshing..." (background poll), green wifi "Updated HH:MM:SS" (live)
- **M%26M / URL-encoding fix** — NSE symbols with & (M&M, M&MFIN) were displaying as M%26M in the stock page header. Fixed with `decodeURIComponent` on the stock page params and `encodeURIComponent` when building `/stock/` URLs in heatmap, context menu, and picks pages
- **Paper Trade modal unblocked** — Buy button was disabled until `fetchPrediction` completed (10–30s). Prediction now runs in background for stop loss/target suggestions only; button is immediately available

Infrastructure:

- **Migrated backend from Render** → **Railway Hobby** (\$5/month, always-on, no cold starts)
- **PICKS_CANDIDATES=751** set in Railway environment — expands the momentum screen candidate pool

Session 4 — 2026-06-19

Forensic Audit & Critical Fixes (9 issues resolved):

- **BUY threshold lowered 70 → 60** — the 70-point threshold was structurally unreachable for most NSE stocks on neutral/bearish market days. New thresholds: BUY ≥ 60, HOLD 45–59, SELL < 45. Score bands updated to match exactly (no more "Good Watchlist Stock" label on stocks treated as HOLD).
- **Fundamental score per-category budgets** — replaced unbounded additive accumulation (theoretical max ~215) with six capped buckets: valuation ±15, profitability ±15, growth ±15, balance sheet ±10, governance ±10, banking ±10. Scores now discriminate meaningfully across the full 0–100 range.
- **Growth double-counting fixed** — revenue and earnings growth were previously counted 3–5× simultaneously (TTM + 3Y CAGR + 5Y CAGR + trend, all additive). Now each counted once via the longest available window; quarterly trend is a capped supplement.
- **Quality gate fixed** — OCF check used Python `or` which treated zero cash flow as falsy; now uses explicit `is None`. Rejection logic split into independent OR conditions (was AND — too strict).
- **Sentiment denominator corrected** — neutral articles no longer dilute bullish signal. Denominator is now bullish + bearish only.
- **Race condition on `_generating flag`** — concurrent POST `/picks/generate` requests could both pass the guard simultaneously. Fixed with `threading.Lock`.
- **Optimizer fallback `max_weight` enforcement** — the fallback could allocate 100% to one stock when alphas were skewed. Fixed with iterative clipping loop.
- **Long-horizon IC training** — long horizon was training on 20D returns (1 month) despite a 3-6 month stated horizon. Now uses `return_60d`. Outcome logger waits 90 calendar days before resolving long-horizon predictions.
- **Partial returns never logged** — outcome logger previously logged partial forward returns when the window hadn't elapsed, corrupting IC training data. Now returns `None` until the full window is complete.
- **Validation thresholds synced** — validation BUY threshold (was 65) now matches live system (60) for all horizons.
- **`picks_generated_today()` logic fixed** — a prior run that produced 0 BUY signals (before threshold fix) saved an empty payload with today's date, causing the startup catch-up to skip regeneration. Now requires at least one actual pick to count as "done today".

- **NSE Daily Picks expanded** — universe expanded from Nifty 100 (96 stocks) to all NSE-listed stocks screened by market cap $\geq$ ₹100 Cr (~500-600 stocks). Two-phase pipeline: Phase-0 bulk momentum screen → Phase-1 deep prediction on top 50 candidates only (memory-safe batching of 300).
- **Documentation fully updated** — README and STOCKSENSE_DOCUMENTATION.md updated to reflect all threshold, formula, and architecture changes.

Live test results (2026-06-19): - /health: ■ ok - /api/predictions/TCS?market=IN&horizon=medium: ■ Score 78, BUY, Strong Buy Candidate - /api/screener/top-movers?market=IN: ■ 10 gainers, 10 losers - /api/validation/results?horizon=medium: ■ Hit rate 56.6%, avg return 3.75% - /api/picks/status: ■ Generating (startup catch-up triggered correctly)

Session 3 — 2026-06-18

New Features: - **Paper Trading module** — full simulated trading with open/close positions, stop-loss/target tracking, unrealised and realised P&L, Postgres persistence - **Price Alerts system** — Postgres-backed price level alerts with live trigger detection - **Daily Picks Trust Layer** — real backtest results, confidence calibration table per score band, live P&L tracker for past picks - **Pick Card UI overhaul** — rank badges (#1–#5), sector tags, top 3 signals visible inline, compact regime bar

Fixes: - Watchlist migrated from ephemeral JSON file to Postgres — no longer disappears on Render restart - screener.in login now fires at startup (not lazily) — data available from first request - screener.in login enhanced: tries both `username/email` field names, logs every step - Daily picks cron now waits for Render to be healthy before triggering — prevents silent cold-start failures - Paper trading open positions: stop loss/target hint row now always visible (shows "not set — click ↗ to add one" when unset, for consistent layout) - Open positions now always show SL/target hint row consistently regardless of signal type

Performance Improvements: - `get_fundamentals`: replaced `3×sleep(3)` blocking retry with `asyncio.wait_for(timeout=8s)` - OHLCV data: added 5-minute in-process cache (was completely uncached) - Quote enrichment: removed redundant second `yfinance fast_info` call - Heatmap cache TTL: 3 min → 5 min - Dashboard refetch interval: 60s → 120s, `staleTime` aligned - Heatmap frontend refetch: 3 min → 5 min, `staleTime` aligned - Stock detail quote refetch: 30s → 60s - `refetchOnWindowFocus: false` added across all major queries

Session 2 — (prior)

- Walk-forward validation engine with Postgres storage
- Learning Alpha Engine (IC weights, regime clustering, meta-model, outcome logger)
- Screener.in authenticated scraping for Indian fundamental data
- Portfolio page with BUY/HOLD/SELL signals per holding
- History tab with horizon selector on portfolio page
- Backtest async fix (non-blocking event loop)
- Prediction cache size cap (300 entries, LRU eviction)
- Hammer & Morning Star candlestick pattern bug fixes

Session 1 — (initial)

- Core prediction engine (technical, fundamental, sentiment, quality, macro)
- Stock detail page with full factor breakdown
- Dashboard with top movers (US/IN/Crypto)
- Heatmap page (sector-wise colour-coded)
- Screener with filters

- Watchlist with live prices
- Daily Picks engine (9-phase Learning Alpha pipeline)
- GitHub Actions automation (daily picks, weekly validation, keep-alive)
- Render + Vercel deployment

This document is a living record of StockSense. It is updated with every significant change to the product.